libgpiodcxx

2.2.4

Generated by Doxygen 1.9.8

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 5

Namespace Documentation

5.1 gpiod::line Namespace Reference

Namespace containing various type definitions for GPIO lines.

Classes

- class offset

    Wrapper around unsigned int for representing line offsets.

Typedefs

- using **offsets** = ::std::vector< offset >

    Vector of line offsets.
- using **values** = ::std::vector< value >

    Vector of line values.
- using **value_mapping** = ::std::pair< offset, value >

    Represents a mapping of a line offset to line logical state.
- using **value_mappings** = ::std::vector< value_mapping >

    Vector of offset->value mappings. Each mapping is defined as a pair of an unsigned and signed integers.

Enumerations

- enum class value { INACTIVE = 0 , ACTIVE = 1 }

    Logical line states.
- enum class direction { AS_IS = 1 , INPUT , OUTPUT }

    Direction settings.
- enum class edge { NONE = 1 , RISING , FALLING , BOTH }

    Edge detection settings.
- enum class bias {
  AS_IS = 1 , UNKNOWN , DISABLED , PULL_UP ,
  PULL_DOWN }

    Internal bias settings.
- enum class drive { PUSH_PULL = 1 , OPEN_DRAIN , OPEN_SOURCE }

    Drive settings.
- enum class clock { MONOTONIC = 1 , REALTIME , **HTE** }

    Event clock settings.

Functions

- ::std::ostream & operator<< (::std::ostream &out, value val)

  Stream insertion operator for logical line values.
- ::std::ostream & operator<< (::std::ostream &out, direction dir)

  Stream insertion operator for direction values.
- ::std::ostream & operator<< (::std::ostream &out, edge edge)

  Stream insertion operator for edge detection values.
- ::std::ostream & operator<< (::std::ostream &out, bias bias)

  Stream insertion operator for bias values.
- ::std::ostream & operator<< (::std::ostream &out, drive drive)

  Stream insertion operator for drive values.
- ::std::ostream & operator<< (::std::ostream &out, clock clock)

  Stream insertion operator for event clock values.
- ::std::ostream & operator<< (::std::ostream &out, const values &vals)

  Stream insertion operator for the list of output values.
- ::std::ostream & operator<< (::std::ostream &out, const offsets &offs)

  Stream insertion operator for the list of line offsets.
- ::std::ostream & operator<< (::std::ostream &out, const value_mapping &mapping)

  Stream insertion operator for the offset-to-value mapping.
- ::std::ostream & operator<< (::std::ostream &out, const value_mappings &mappings)

  Stream insertion operator for the list of offset-to-value mappings.
- template<typename T >
  ::std::ostream & **insert_vector** (::std::ostream &out, const ::std::string &name, const ::std::vector< T > &vec)

5.1.1 Detailed Description

Namespace containing various type definitions for GPIO lines.

5.1.2 Enumeration Type Documentation

5.1.2.1 bias

```
enum class gpiod::line::bias  [strong]
```

Internal bias settings.

Enumerator

AS_IS	Don't change the bias setting when applying line config.
UNKNOWN	The internal bias state is unknown.
DISABLED	The internal bias is disabled.
PULL_UP	The internal pull-up bias is enabled.
PULL_DOWN	The internal pull-down bias is enabled.

5.1.2.2 clock

```
enum class gpiod::line::clock  [strong]
```

Event clock settings.

Enumerator

MONOTONIC	Line uses the monotonic clock for edge event timestamps.
REALTIME	Line uses the realtime clock for edge event timestamps.

5.1.2.3 direction

`enum class gpiod::line::direction [strong]`

Direction settings.

Enumerator

AS_IS	Request the line(s), but don't change current direction.
INPUT	Direction is input - we're reading the state of a GPIO line.
OUTPUT	Direction is output - we're driving the GPIO line.

5.1.2.4 drive

`enum class gpiod::line::drive [strong]`

Drive settings.

Enumerator

PUSH_PULL	Drive setting is push-pull.
OPEN_DRAIN	Line output is open-drain.
OPEN_SOURCE	Line output is open-source.

5.1.2.5 edge

`enum class gpiod::line::edge [strong]`

Edge detection settings.

Enumerator

NONE	Line edge detection is disabled.
RISING	Line detects rising edge events.
FALLING	Line detect falling edge events.
BOTH	Line detects both rising and falling edge events.

5.1.2.6 value

```
enum class gpiod::line::value [strong]
```

Logical line states.

Enumerator

INACTIVE	Line is inactive.
ACTIVE	Line is active.

5.1.3 Function Documentation

5.1.3.1 operator<<() [1/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::bias bias )
```

Stream insertion operator for bias values.

Parameters

out	Output stream.
bias	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.2 operator<<() [2/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::clock clock )
```

Stream insertion operator for event clock values.

Parameters

out	Output stream.
clock	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.3 operator<<() [3/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            const offsets & offs )
```

Stream insertion operator for the list of line offsets.

Parameters

out	Output stream.
offs	Object to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.4 operator<<() [4/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            const value_mapping & mapping )
```

Stream insertion operator for the offset-to-value mapping.

Parameters

out	Output stream.
mapping	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.5 operator<<() [5/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            const value_mappings & mappings )
```

Stream insertion operator for the list of offset-to-value mappings.

Parameters

out	Output stream.
mappings	Object to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.6 operator<<() [6/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            const values & vals )
```

Stream insertion operator for the list of output values.

Parameters

out	Output stream.
vals	Object to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.7 operator<<() [7/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::direction dir )
```

Stream insertion operator for direction values.

Parameters

out	Output stream.
dir	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.8 operator<<() [8/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::drive drive )
```

Stream insertion operator for drive values.

Parameters

out	Output stream.
drive	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.9 operator<<() [9/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::edge edge )
```

Stream insertion operator for edge detection values.

Parameters

out	Output stream.
edge	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

5.1.3.10 operator<<() [10/10]

```
GPIOD_CXX_API::std::ostream & gpiod::line::operator<< (
            ::std::ostream & out,
            line::value val )
```

Stream insertion operator for logical line values.

Parameters

out	Output stream.
val	Value to insert into the output stream in a human-readable form.

Returns

Reference to out.

Chapter 6

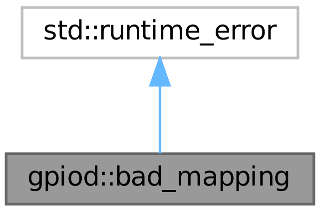
Class Documentation

6.1 gpiod::bad_mapping Class Reference

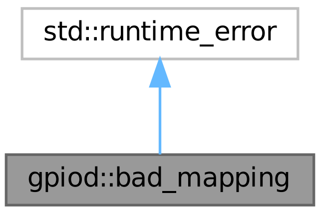
Exception thrown when the core C library returns an invalid value for any of the line_info properties.

`#include <exception.hpp>`

Inheritance diagram for gpiod::bad_mapping:



Collaboration diagram for gpiod::bad_mapping:



Public Member Functions

- bad_mapping (const ::std::string &what)
  Constructor.
- bad_mapping (const bad_mapping &other) noexcept
  Copy constructor.
- bad_mapping (bad_mapping &&other) noexcept
  Move constructor.
- bad_mapping & operator= (const bad_mapping &other) noexcept
  Assignment operator.
- bad_mapping & operator= (bad_mapping &&other) noexcept
  Move assignment operator.

6.1.1 Detailed Description

Exception thrown when the core C library returns an invalid value for any of the line_info properties.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 bad_mapping() [1/3]

```
GPIOD_CXX_API gpiod::bad_mapping::bad_mapping (
            const ::std::string & what )  [explicit]
```

Constructor.

Parameters

what	Human readable reason for error.

6.1.2.2 bad_mapping() [2/3]

```
GPIOD_CXX_API gpiod::bad_mapping::bad_mapping (
            const bad_mapping & other )  [noexcept]
```

Copy constructor.

Parameters

other	Object to copy from.

6.1.2.3 bad_mapping() [3/3]

```
GPIOD_CXX_API gpiod::bad_mapping::bad_mapping (
            bad_mapping && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.1.3 Member Function Documentation

6.1.3.1 operator=() [1/2]

```
GPIOD_CXX_API bad_mapping & gpiod::bad_mapping::operator= (
            bad_mapping && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.1.3.2 operator=() [2/2]

```
GPIOD_CXX_API bad_mapping & gpiod::bad_mapping::operator= (
            const bad_mapping & other )  [noexcept]
```

Assignment operator.

Parameters

other	Object to copy from.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/exception.hpp
- src/exception.cpp

6.2 gpiod::chip Class Reference

Represents a GPIO chip.

`#include <chip.hpp>`

Classes

- struct impl

Public Member Functions

- chip (const ::std::filesystem::path &path)

    Instantiates a new chip object by opening the device file indicated by the `path` *argument.*
- chip (chip &&other) noexcept

    Move constructor.
- chip & **operator=** (const chip &other)=delete
- chip & operator= (chip &&other) noexcept

    Move assignment operator.
- operator bool () const noexcept

    Check if this object is valid.
- void close ()

    Close the GPIO chip device file and free associated resources.
- ::std::filesystem::path path () const

    Get the filesystem path that was used to open this GPIO chip.
- chip_info get_info () const

    Get information about the chip.
- line_info get_line_info (line::offset offset) const

    Retrieve the current snapshot of line information for a single line.
- line_info watch_line_info (line::offset offset) const

    Wrapper around gpiod::chip::get_line_info that retrieves the line info and starts watching the line for changes.
- void unwatch_line_info (line::offset offset) const

    Stop watching the line at given offset for info events.
- int fd () const

    Get the file descriptor associated with this chip.
- bool wait_info_event (const ::std::chrono::nanoseconds &timeout) const

    Wait for line status events on any of the watched lines exposed by this chip.
- info_event read_info_event () const

    Read a single line status change event from this chip.
- int get_line_offset_from_name (const ::std::string &name) const

    Map a GPIO line's name to its offset within the chip.
- request_builder prepare_request ()

    Create a request_builder associated with this chip.

6.2.1 Detailed Description

Represents a GPIO chip.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 chip() [1/2]

```
GPIOD_CXX_API gpiod::chip::chip (
            const ::std::filesystem::path & path )  [explicit]
```

Instantiates a new chip object by opening the device file indicated by the `path` argument.

Parameters

path	Path to the device file to open.

6.2.2.2 chip() [2/2]

```
GPIOD_CXX_API gpiod::chip::chip (
            chip && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.2.3 Member Function Documentation

6.2.3.1 close()

```
GPIOD_CXX_API void gpiod::chip::close ( )
```

Close the GPIO chip device file and free associated resources.

Note

The chip object can live after calling this method but any of the chip's mutators will throw a logic_error exception.

6.2.3.2 fd()

```
GPIOD_CXX_API int gpiod::chip::fd ( ) const
```

Get the file descriptor associated with this chip.

Returns

File descriptor number.

6.2.3.3 get_info()

```
GPIOD_CXX_API chip_info gpiod::chip::get_info ( ) const
```

Get information about the chip.

Returns

New chip_info object.

6.2.3.4 get_line_info()

```
GPIOD_CXX_API line_info gpiod::chip::get_line_info (
            line::offset offset ) const
```

Retrieve the current snapshot of line information for a single line.

Parameters

offset	Offset of the line to get the info for.

Returns

New gpiod::line_info object.

6.2.3.5 get_line_offset_from_name()

```
GPIOD_CXX_API int gpiod::chip::get_line_offset_from_name (
            const ::std::string & name ) const
```

Map a GPIO line's name to its offset within the chip.

Parameters

name	Name of the GPIO line to map.

Returns

Offset of the line within the chip or -1 if the line with given name is not exposed by this chip.

6.2.3.6 operator bool()

```
GPIOD_CXX_API gpiod::chip::operator bool ( ) const  [explicit], [noexcept]
```

Check if this object is valid.

Returns

True if this object's methods can be used, false otherwise. False usually means the chip was closed. If the user calls any of the methods of this class on an object for which this operator returned false, a logic_error will be thrown.

6.2.3.7 operator=()

```
GPIOD_CXX_API chip & gpiod::chip::operator= (
            chip && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.2.3.8 path()

```
GPIOD_CXX_API::std::filesystem::path gpiod::chip::path ( ) const
```

Get the filesystem path that was used to open this GPIO chip.

Returns

Path to the underlying character device file.

6.2.3.9 prepare_request()

```
GPIOD_CXX_API request_builder gpiod::chip::prepare_request ( )
```

Create a request_builder associated with this chip.

Returns

New request_builder object.

6.2.3.10 read_info_event()

```
GPIOD_CXX_API info_event gpiod::chip::read_info_event ( ) const
```

Read a single line status change event from this chip.

Returns

New info_event object.

6.2.3.11 unwatch_line_info()

```
GPIOD_CXX_API void gpiod::chip::unwatch_line_info (
            line::offset offset ) const
```

Stop watching the line at given offset for info events.

Parameters

offset	Offset of the line to get the info for.

6.2.3.12 wait_info_event()

```
GPIOD_CXX_API bool gpiod::chip::wait_info_event (
            const ::std::chrono::nanoseconds & timeout ) const
```

Wait for line status events on any of the watched lines exposed by this chip.

Parameters

timeout	Wait time limit in nanoseconds. If set to 0, the function returns immediately. If set to a negative number, the function blocks indefinitely until an event becomes available.

Returns

True if at least one event is ready to be read. False if the wait timed out.

6.2.3.13 watch_line_info()

```
GPIOD_CXX_API line_info gpiod::chip::watch_line_info (
            line::offset offset ) const
```

Wrapper around gpiod::chip::get_line_info that retrieves the line info and starts watching the line for changes.

Parameters

offset	Offset of the line to get the info for.

Returns

New gpiod::line_info object.

The documentation for this class was generated from the following files:

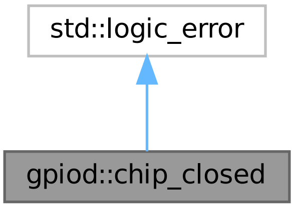
- src/gpiodcxx/chip.hpp
- src/chip.cpp

6.3 gpiod::chip_closed Class Reference

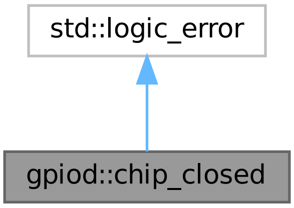
Exception thrown when an already closed chip is used.

```
#include <exception.hpp>
```

Inheritance diagram for gpiod::chip_closed:



Collaboration diagram for gpiod::chip_closed:



Public Member Functions

- chip_closed (const ::std::string &what)
  Constructor.
- chip_closed (const chip_closed &other) noexcept
  Copy constructor.
- chip_closed (chip_closed &&other) noexcept
  Move constructor.
- chip_closed & operator= (const chip_closed &other) noexcept
  Assignment operator.
- chip_closed & operator= (chip_closed &&other) noexcept
  Move assignment operator.

6.3.1 Detailed Description

Exception thrown when an already closed chip is used.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 chip_closed() [1/3]

```
GPIOD_CXX_API gpiod::chip_closed::chip_closed (
            const ::std::string & what ) [explicit]
```

Constructor.

Parameters

what	Human readable reason for error.

6.3.2.2 chip_closed() [2/3]

```
GPIOD_CXX_API gpiod::chip_closed::chip_closed (
            const chip_closed & other ) [noexcept]
```

Copy constructor.

Parameters

other	Object to copy from.

6.3.2.3 chip_closed() [3/3]

```
GPIOD_CXX_API gpiod::chip_closed::chip_closed (
            chip_closed && other ) [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.3.3 Member Function Documentation

6.3.3.1 operator=() [1/2]

```
GPIOD_CXX_API chip_closed & gpiod::chip_closed::operator= (
            chip_closed && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.3.3.2 operator=() [2/2]

```
GPIOD_CXX_API chip_closed & gpiod::chip_closed::operator= (
            const chip_closed & other )  [noexcept]
```

Assignment operator.

Parameters

other	Object to copy from.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/exception.hpp
- src/exception.cpp

6.4 gpiod::chip_info Class Reference

Represents an immutable snapshot of GPIO chip information.

`#include <chip-info.hpp>`

Classes

- struct impl

Public Member Functions

- chip_info (const chip_info &other)
  Copy constructor.
- chip_info (chip_info &&other) noexcept
  Move constructor.
- chip_info & operator= (const chip_info &other)
  Assignment operator.
- chip_info & operator= (chip_info &&other) noexcept
  Move assignment operator.
- ::std::string name () const noexcept
  Get the name of this GPIO chip.
- ::std::string label () const noexcept
  Get the label of this GPIO chip.
- ::std::size_t num_lines () const noexcept
  Return the number of lines exposed by this chip.

6.4.1 Detailed Description

Represents an immutable snapshot of GPIO chip information.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 chip_info() [1/2]

```
GPIOD_CXX_API gpiod::chip_info::chip_info (
            const chip_info & other )
```

Copy constructor.

Parameters

other	Object to copy.

6.4.2.2 chip_info() [2/2]

```
GPIOD_CXX_API gpiod::chip_info::chip_info (
            chip_info && other ) [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.4.3 Member Function Documentation

6.4.3.1 label()

```
GPIOD_CXX_API::std::string gpiod::chip_info::label ( ) const [noexcept]
```

Get the label of this GPIO chip.

Returns

String containing the chip name.

6.4.3.2 name()

```
GPIOD_CXX_API::std::string gpiod::chip_info::name ( ) const [noexcept]
```

Get the name of this GPIO chip.

Returns

String containing the chip name.

6.4.3.3 num_lines()

```
GPIOD_CXX_API::std::size_t gpiod::chip_info::num_lines ( ) const  [noexcept]
```

Return the number of lines exposed by this chip.

Returns

Number of lines.

6.4.3.4 operator=() [1/2]

```
GPIOD_CXX_API chip_info & gpiod::chip_info::operator= (
            chip_info && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.4.3.5 operator=() [2/2]

```
GPIOD_CXX_API chip_info & gpiod::chip_info::operator= (
            const chip_info & other )
```

Assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/chip-info.hpp
- src/chip-info.cpp

6.5 gpiod::deleter< T, F > Struct Template Reference

Public Member Functions

- void **operator()** (T ∗ptr)

The documentation for this struct was generated from the following file:

- src/include/internal.hpp

6.6 gpiod::edge_event Class Reference

Immutable object containing data about a single edge event.

```
#include <edge-event.hpp>
```

Classes

- struct impl
- struct impl_external
- struct impl_managed

Public Types

- enum class event_type { RISING_EDGE = 1 , FALLING_EDGE }

  Edge event types.

Public Member Functions

- edge_event (const edge_event &other)

  Copy constructor.
- edge_event (edge_event &&other) noexcept

  Move constructor.
- edge_event & operator= (const edge_event &other)

  Copy assignment operator.
- edge_event & operator= (edge_event &&other) noexcept

  Move assignment operator.
- event_type type () const

  Retrieve the event type.
- timestamp timestamp_ns () const noexcept

  Retrieve the event time-stamp.
- line::offset line_offset () const noexcept

  Read the offset of the line on which this event was registered.
- unsigned long global_seqno () const noexcept

  Get the global sequence number of this event.
- unsigned long line_seqno () const noexcept

  Get the event sequence number specific to the concerned line.

6.6.1 Detailed Description

Immutable object containing data about a single edge event.

6.6.2 Member Enumeration Documentation

6.6.2.1 event_type

```
enum class gpiod::edge_event::event_type  [strong]
```

Edge event types.

Enumerator

RISING_EDGE	This is a rising edge event.
FALLING_EDGE	This is falling edge event.

6.6.3 Constructor & Destructor Documentation

6.6.3.1 edge_event() [1/2]

```
GPIOD_CXX_API gpiod::edge_event::edge_event (
            const edge_event & other )
```

Copy constructor.

Parameters

other	Object to copy.

6.6.3.2 edge_event() [2/2]

```
GPIOD_CXX_API gpiod::edge_event::edge_event (
            edge_event && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.6.4 Member Function Documentation

6.6.4.1 global_seqno()

```
GPIOD_CXX_API unsigned long gpiod::edge_event::global_seqno ( ) const [noexcept]
```

Get the global sequence number of this event.

Returns

Sequence number of the event relative to all lines in the associated line request.

6.6.4.2 line_offset()

```
GPIOD_CXX_API line::offset gpiod::edge_event::line_offset ( ) const [noexcept]
```

Read the offset of the line on which this event was registered.

Returns

Line offset.

6.6.4.3 line_seqno()

```
GPIOD_CXX_API unsigned long gpiod::edge_event::line_seqno ( ) const [noexcept]
```

Get the event sequence number specific to the concerned line.

Returns

Sequence number of the event relative to this line within the lifetime of the associated line request.

6.6.4.4 operator=() [1/2]

```
GPIOD_CXX_API edge_event & gpiod::edge_event::operator= (
            const edge_event & other )
```

Copy assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.6.4.5 operator=() [2/2]

```
GPIOD_CXX_API edge_event & gpiod::edge_event::operator= (
            edge_event && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.6.4.6 timestamp_ns()

```
GPIOD_CXX_API timestamp gpiod::edge_event::timestamp_ns ( ) const [noexcept]
```

Retrieve the event time-stamp.

Returns

Time-stamp in nanoseconds as registered by the kernel using the configured edge event clock.

6.6.4.7 type()

```
GPIOD_CXX_API edge_event::event_type gpiod::edge_event::type ( ) const
```

Retrieve the event type.

Returns

Event type (rising or falling edge).

The documentation for this class was generated from the following files:

- src/gpiodcxx/edge-event.hpp
- src/edge-event.cpp

6.7 gpiod::edge_event_buffer Class Reference

Object into which edge events are read for better performance.

```
#include <edge-event-buffer.hpp>
```

Classes

- struct impl

Public Types

- using **const_iterator** = ::std::vector< edge_event >::const_iterator

  Constant iterator for iterating over edge events stored in the buffer.

Public Member Functions

- edge_event_buffer (::std::size_t capacity=64)

  Constructor. Creates a new edge event buffer with given capacity.
- **edge_event_buffer** (const edge_event_buffer &other)=delete
- edge_event_buffer (edge_event_buffer &&other) noexcept

  Move constructor.
- edge_event_buffer & **operator=** (const edge_event_buffer &other)=delete
- edge_event_buffer & operator= (edge_event_buffer &&other) noexcept

  Move assignment operator.
- const edge_event & get_event (unsigned int index) const

  Get the constant reference to the edge event at given index.
- ::std::size_t num_events () const

  Get the number of edge events currently stored in the buffer.
- ::std::size_t capacity () const noexcept

  Maximum capacity of the buffer.
- const_iterator begin () const noexcept

  Get a constant iterator to the first edge event currently stored in the buffer.
- const_iterator end () const noexcept

  Get a constant iterator to the element after the last edge event in the buffer.

6.7.1 Detailed Description

Object into which edge events are read for better performance.

The edge_event_buffer allows reading edge_event objects into an existing buffer which improves the performance by avoiding needless memory allocations.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 edge_event_buffer() [1/2]

```
GPIOD_CXX_API gpiod::edge_event_buffer::edge_event_buffer (
            ::std::size_t capacity = 64 )  [explicit]
```

Constructor. Creates a new edge event buffer with given capacity.

Parameters

capacity	Capacity of the new buffer.

6.7.2.2 edge_event_buffer() [2/2]

```
GPIOD_CXX_API gpiod::edge_event_buffer::edge_event_buffer (
            edge_event_buffer && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.7.3 Member Function Documentation

6.7.3.1 begin()

```
GPIOD_CXX_API edge_event_buffer::const_iterator gpiod::edge_event_buffer::begin ( ) const
[noexcept]
```

Get a constant iterator to the first edge event currently stored in the buffer.

Returns

Constant iterator to the first element.

6.7.3.2 capacity()

```
GPIOD_CXX_API::std::size_t gpiod::edge_event_buffer::capacity ( ) const  [noexcept]
```

Maximum capacity of the buffer.

Returns

Buffer capacity.

6.7.3.3 end()

```
GPIOD_CXX_API edge_event_buffer::const_iterator gpiod::edge_event_buffer::end ( ) const  [noexcept]
```

Get a constant iterator to the element after the last edge event in the buffer.

Returns

Constant iterator to the element after the last edge event.

6.7.3.4 get_event()

```
GPIOD_CXX_API const edge_event & gpiod::edge_event_buffer::get_event (
            unsigned int index ) const
```

Get the constant reference to the edge event at given index.

Parameters

index	Index of the event in the buffer.

Returns

Constant reference to the edge event.

6.7.3.5 num_events()

```
GPIOD_CXX_API::std::size_t gpiod::edge_event_buffer::num_events ( ) const
```

Get the number of edge events currently stored in the buffer.

Returns

Number of edge events in the buffer.

6.7.3.6 operator=()

```
GPIOD_CXX_API edge_event_buffer & gpiod::edge_event_buffer::operator= (
            edge_event_buffer && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/edge-event-buffer.hpp
- src/edge-event-buffer.cpp

6.8 gpiod::chip::impl Struct Reference

Public Member Functions

- **impl** (const ::std::filesystem::path &path)
- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- void **throw_if_closed** () const

Public Attributes

- chip_ptr **chip**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/chip.cpp

6.9 gpiod::chip_info::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- void **set_info_ptr** (chip_info_ptr &new_info)

Public Attributes

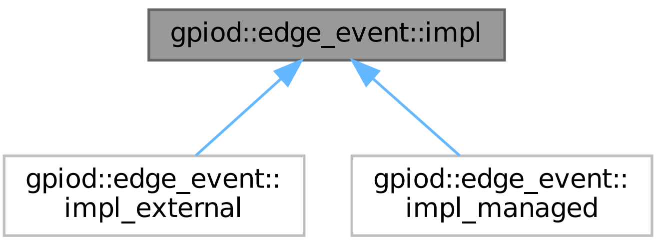
- chip_info_ptr **info**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/chip-info.cpp

6.10 gpiod::edge_event::impl Struct Reference

Inheritance diagram for gpiod::edge_event::impl:



Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- virtual ::gpiod_edge_event ∗ **get_event_ptr** () const noexcept=0
- virtual ::std::shared_ptr< impl > **copy** (const ::std::shared_ptr< impl > &self) const =0

The documentation for this struct was generated from the following file:

- src/include/internal.hpp

6.11 gpiod::edge_event_buffer::impl Struct Reference

Public Member Functions

- **impl** (unsigned int capacity)
- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- int **read_events** (const line_request_ptr &request, unsigned int max_events)

Public Attributes

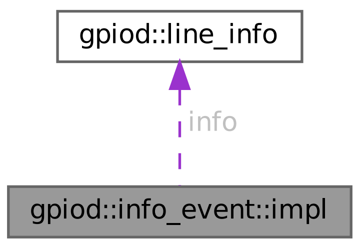
- edge_event_buffer_ptr **buffer**
- ::std::vector< edge_event > **events**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/edge-event-buffer.cpp

6.12 gpiod::info_event::impl Struct Reference

Collaboration diagram for gpiod::info_event::impl:



Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- void **set_info_event_ptr** (info_event_ptr &new_event)

Public Attributes

- info_event_ptr **event**
- line_info **info**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/info-event.cpp

6.13 gpiod::line_config::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

- line_config_ptr **config**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/line-config.cpp

6.14 gpiod::line_info::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- void **set_info_ptr** (line_info_ptr &new_info)

Public Attributes

- line_info_ptr **info**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/line-info.cpp

6.15 gpiod::line_request::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete
- void **throw_if_released** () const
- void **set_request_ptr** (line_request_ptr &ptr)
- void **fill_offset_buf** (const line::offsets &offsets)

Public Attributes

- line_request_ptr **request**
- ::std::vector< unsigned int > **offset_buf**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/line-request.cpp

6.16 gpiod::line_settings::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

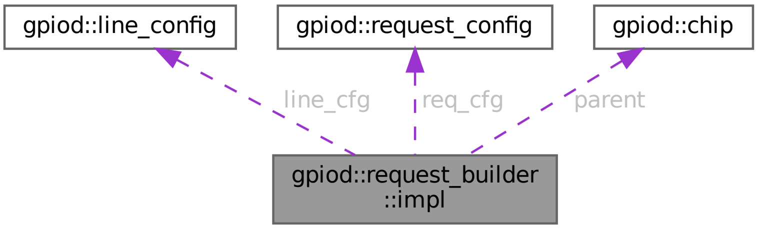
- line_settings_ptr **settings**

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/line-settings.cpp

6.17 gpiod::request_builder::impl Struct Reference

Collaboration diagram for gpiod::request_builder::impl:



Public Member Functions

- **impl** (chip &parent)
- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

- line_config **line_cfg**
- request_config **req_cfg**
- chip **parent**

The documentation for this struct was generated from the following file:

- src/request-builder.cpp

6.18 gpiod::request_config::impl Struct Reference

Public Member Functions

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

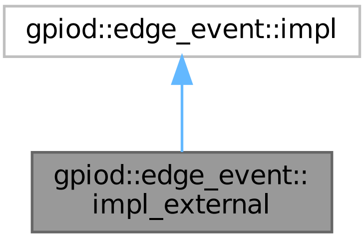
- request_config_ptr **config**

The documentation for this struct was generated from the following files:

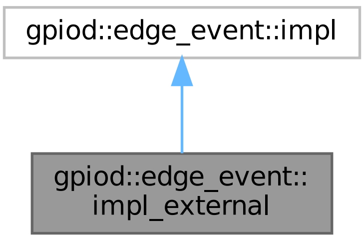
- src/include/internal.hpp
- src/request-config.cpp

6.19 gpiod::edge_event::impl_external Struct Reference

Inheritance diagram for gpiod::edge_event::impl_external:



Collaboration diagram for gpiod::edge_event::impl_external:



Public Member Functions

- **impl_external** (const impl_external &other)=delete
- **impl_external** (impl_external &&other)=delete
- impl_external & **operator=** (const impl_external &other)=delete
- impl_external & **operator=** (impl_external &&other)=delete
- ::gpiod_edge_event ∗ get_event_ptr () const noexcept override
- ::std::shared_ptr< impl > copy (const ::std::shared_ptr< impl > &self) const override

Public Member Functions inherited from gpiod::edge_event::impl

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

- ::gpiod_edge_event ∗ **event**

6.19.1 Member Function Documentation

6.19.1.1 copy()

```
std::shared_ptr< edge_event::impl > gpiod::edge_event::impl_external::copy (
            const ::std::shared_ptr< impl > & self ) const [override], [virtual]
```

Implements gpiod::edge_event::impl.

6.19.1.2 get_event_ptr()

```
gpiod_edge_event * gpiod::edge_event::impl_external::get_event_ptr ( ) const [override], [virtual],
[noexcept]
```

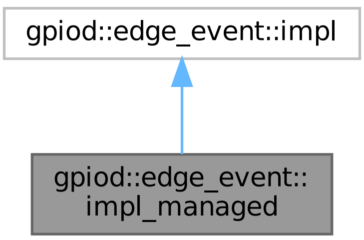
Implements gpiod::edge_event::impl.

The documentation for this struct was generated from the following files:

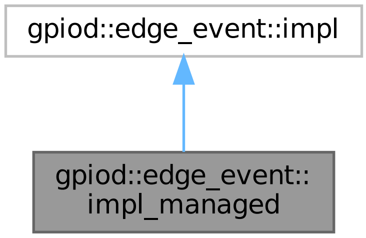
- src/include/internal.hpp
- src/edge-event.cpp

6.20 gpiod::edge_event::impl_managed Struct Reference

Inheritance diagram for gpiod::edge_event::impl_managed:



Collaboration diagram for gpiod::edge_event::impl_managed:



Public Member Functions

- **impl_managed** (const impl_managed &other)=delete
- **impl_managed** (impl_managed &&other)=delete
- impl_managed & **operator=** (const impl_managed &other)=delete
- impl_managed & **operator=** (impl_managed &&other)=delete
- ::gpiod_edge_event ∗ get_event_ptr () const noexcept override
- ::std::shared_ptr< impl > copy (const ::std::shared_ptr< impl > &self) const override

Public Member Functions inherited from gpiod::edge_event::impl

- **impl** (const impl &other)=delete
- **impl** (impl &&other)=delete
- impl & **operator=** (const impl &other)=delete
- impl & **operator=** (impl &&other)=delete

Public Attributes

- edge_event_ptr **event**

6.20.1 Member Function Documentation

6.20.1.1 copy()

```
std::shared_ptr< edge_event::impl > gpiod::edge_event::impl_managed::copy (
            const ::std::shared_ptr< impl > & self ) const [override], [virtual]
```

Implements gpiod::edge_event::impl.

6.20.1.2 get_event_ptr()

```
gpiod_edge_event * gpiod::edge_event::impl_managed::get_event_ptr ( ) const  [override], [virtual], [noexcept]
```

Implements gpiod::edge_event::impl.

The documentation for this struct was generated from the following files:

- src/include/internal.hpp
- src/edge-event.cpp

6.21 gpiod::info_event Class Reference

Immutable object containing data about a single line info event.

`#include <info-event.hpp>`

Classes

- struct impl

Public Types

- enum class event_type { LINE_REQUESTED = 1 , LINE_RELEASED , LINE_CONFIG_CHANGED }
  Types of info events.

Public Member Functions

- info_event (const info_event &other)
  Copy constructor.
- info_event (info_event &&other) noexcept
  Move constructor.
- info_event & operator= (const info_event &other)
  Copy assignment operator.
- info_event & operator= (info_event &&other) noexcept
  Move assignment operator.
- event_type type () const
  Type of this event.
- ::std::uint64_t timestamp_ns () const noexcept
  Timestamp of the event as returned by the kernel.
- const line_info & get_line_info () const noexcept
  Get the new line information.

6.21.1 Detailed Description

Immutable object containing data about a single line info event.

6.21.2 Member Enumeration Documentation

6.21.2.1 event_type

```
enum class gpiod::info_event::event_type  [strong]
```

Types of info events.

Enumerator

LINE_REQUESTED	Line has been requested.
LINE_RELEASED	Previously requested line has been released.
LINE_CONFIG_CHANGED	Line configuration has changed.

6.21.3 Constructor & Destructor Documentation

6.21.3.1 info_event() [1/2]

```
GPIOD_CXX_API gpiod::info_event::info_event (
            const info_event & other )
```

Copy constructor.

Parameters

other	Object to copy.

6.21.3.2 info_event() [2/2]

```
GPIOD_CXX_API gpiod::info_event::info_event (
            info_event && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.21.4 Member Function Documentation

6.21.4.1 get_line_info()

```
GPIOD_CXX_API const line_info & gpiod::info_event::get_line_info ( ) const  [noexcept]
```

Get the new line information.

Returns

Constant reference to the line info object containing the line data as read at the time of the info event.

6.21.4.2 operator=() [1/2]

```
GPIOD_CXX_API info_event & gpiod::info_event::operator= (
            const info_event & other )
```

Copy assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.21.4.3 operator=() [2/2]

```
GPIOD_CXX_API info_event & gpiod::info_event::operator= (
            info_event && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.21.4.4 timestamp_ns()

```
GPIOD_CXX_API::std::uint64_t gpiod::info_event::timestamp_ns ( ) const [noexcept]
```

Timestamp of the event as returned by the kernel.

Returns

Timestamp as a 64-bit unsigned integer.

6.21.4.5 type()

```
GPIOD_CXX_API info_event::event_type gpiod::info_event::type ( ) const
```

Type of this event.

Returns

Event type.

The documentation for this class was generated from the following files:

- src/gpiodcxx/info-event.hpp
- src/info-event.cpp

6.22 gpiod::line_config Class Reference

Contains a set of line config options used in line requests and reconfiguration.

```
#include <line-config.hpp>
```

Classes

- struct impl

Public Member Functions

- **line_config** (const line_config &other)=delete
- line_config (line_config &&other) noexcept

    Move constructor.
- line_config & operator= (line_config &&other) noexcept

    Move assignment operator.
- line_config & reset () noexcept

    Reset the line config object.
- line_config & add_line_settings (line::offset offset, const line_settings &settings)

    Add line settings for a single offset.
- line_config & add_line_settings (const line::offsets &offsets, const line_settings &settings)

    Add line settings for a set of offsets.
- line_config & set_output_values (const line::values &values)

    Set output values for a number of lines.
- ::std::map< line::offset, line_settings > get_line_settings () const

    Get a mapping of offsets to line settings stored by this object.

6.22.1 Detailed Description

Contains a set of line config options used in line requests and reconfiguration.

6.22.2 Constructor & Destructor Documentation

6.22.2.1 line_config()

```
GPIOD_CXX_API gpiod::line_config::line_config (
            line_config && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.22.3 Member Function Documentation

6.22.3.1 add_line_settings() [1/2]

```
GPIOD_CXX_API line_config & gpiod::line_config::add_line_settings (
            const line::offsets & offsets,
            const line_settings & settings )
```

Add line settings for a set of offsets.

Parameters

offsets	Offsets for which to add settings.
settings	Line settings to add.

Returns

Reference to self.

6.22.3.2 add_line_settings() [2/2]

```
GPIOD_CXX_API line_config & gpiod::line_config::add_line_settings (
            line::offset offset,
            const line_settings & settings )
```

Add line settings for a single offset.

Parameters

offset	Offset for which to add settings.
settings	Line settings to add.

Returns

Reference to self.

6.22.3.3 get_line_settings()

```
GPIOD_CXX_API::std::map< line::offset, line_settings > gpiod::line_config::get_line_settings (
) const
```

Get a mapping of offsets to line settings stored by this object.

Returns

Map in which keys represent line offsets and values are the settings corresponding with them.

6.22.3.4 operator=()

```
GPIOD_CXX_API line_config & gpiod::line_config::operator= (
            line_config && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.22.3.5 reset()

```
GPIOD_CXX_API line_config & gpiod::line_config::reset ( )  [noexcept]
```

Reset the line config object.

Returns

Reference to self.

6.22.3.6 set_output_values()

```
GPIOD_CXX_API line_config & gpiod::line_config::set_output_values (
            const line::values & values )
```

Set output values for a number of lines.

Parameters

values	Buffer containing the output values.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/line-config.hpp
- src/line-config.cpp

6.23 gpiod::line_info Class Reference

Contains an immutable snapshot of the line's state at the time when the object of this class was instantiated.

`#include <line-info.hpp>`

Classes

- struct impl

Public Member Functions

- line_info (const line_info &other) noexcept
  Copy constructor.
- line_info (line_info &&other) noexcept
  Move constructor.
- line_info & operator= (const line_info &other) noexcept
  Copy assignment operator.
- line_info & operator= (line_info &&other) noexcept
  Move assignment operator.
- line::offset offset () const noexcept
  Get the hardware offset of the line.
- ::std::string name () const noexcept
  Get the GPIO line name.
- bool used () const noexcept
  Check if the line is currently in use.
- ::std::string consumer () const noexcept
  Read the GPIO line consumer name.
- line::direction direction () const
  Read the GPIO line direction setting.
- line::edge edge_detection () const
  Read the current edge detection setting of this line.
- line::bias bias () const
  Read the GPIO line bias setting.
- line::drive drive () const
  Read the GPIO line drive setting.
- bool active_low () const noexcept
  Check if the signal of this line is inverted.
- bool debounced () const noexcept
  Check if this line is debounced (either by hardware or by the kernel software debouncer).
- ::std::chrono::microseconds debounce_period () const noexcept
  Read the current debounce period in microseconds.
- line::clock event_clock () const
  Read the current event clock setting used for edge event timestamps.

6.23.1 Detailed Description

Contains an immutable snapshot of the line's state at the time when the object of this class was instantiated.

6.23.2 Constructor & Destructor Documentation

6.23.2.1 line_info() [1/2]

```
GPIOD_CXX_API gpiod::line_info::line_info (
            const line_info & other )  [noexcept]
```

Copy constructor.

Parameters

other	Object to copy.

6.23.2.2 line_info() [2/2]

```
GPIOD_CXX_API gpiod::line_info::line_info (
            line_info && other ) [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.23.3 Member Function Documentation

6.23.3.1 active_low()

```
GPIOD_CXX_API bool gpiod::line_info::active_low ( ) const [noexcept]
```

Check if the signal of this line is inverted.

Returns

True if this line is "active-low", false otherwise.

6.23.3.2 bias()

```
GPIOD_CXX_API line::bias gpiod::line_info::bias ( ) const
```

Read the GPIO line bias setting.

Returns

Returns BIAS_PULL_UP, BIAS_PULL_DOWN, BIAS_DISABLE or BIAS_UNKNOWN.

6.23.3.3 consumer()

```
GPIOD_CXX_API::std::string gpiod::line_info::consumer ( ) const [noexcept]
```

Read the GPIO line consumer name.

Returns

Name of the GPIO consumer name as it is represented in the kernel. This routine returns an empty string if the line is not used.

6.23.3.4 debounce_period()

```
GPIOD_CXX_API::std::chrono::microseconds gpiod::line_info::debounce_period ( ) const [noexcept]
```

Read the current debounce period in microseconds.

Returns

Current debounce period in microseconds, 0 if the line is not debounced.

6.23.3.5 debounced()

```
GPIOD_CXX_API bool gpiod::line_info::debounced ( ) const [noexcept]
```

Check if this line is debounced (either by hardware or by the kernel software debouncer).

Returns

True if the line is debounced, false otherwise.

6.23.3.6 direction()

```
GPIOD_CXX_API line::direction gpiod::line_info::direction ( ) const
```

Read the GPIO line direction setting.

Returns

Returns DIRECTION_INPUT or DIRECTION_OUTPUT.

6.23.3.7 drive()

```
GPIOD_CXX_API line::drive gpiod::line_info::drive ( ) const
```

Read the GPIO line drive setting.

Returns

Returns DRIVE_PUSH_PULL, DRIVE_OPEN_DRAIN or DRIVE_OPEN_SOURCE.

6.23.3.8 edge_detection()

```
GPIOD_CXX_API line::edge gpiod::line_info::edge_detection ( ) const
```

Read the current edge detection setting of this line.

Returns

Returns EDGE_NONE, EDGE_RISING, EDGE_FALLING or EDGE_BOTH.

6.23.3.9 event_clock()

```
GPIOD_CXX_API line::clock gpiod::line_info::event_clock ( ) const
```

Read the current event clock setting used for edge event timestamps.

Returns

Returns MONOTONIC, REALTIME or HTE.

6.23.3.10 name()

```
GPIOD_CXX_API::std::string gpiod::line_info::name ( ) const [noexcept]
```

Get the GPIO line name.

Returns

Name of the GPIO line as it is represented in the kernel. This routine returns an empty string if the line is unnamed.

6.23.3.11 offset()

```
GPIOD_CXX_API line::offset gpiod::line_info::offset ( ) const [noexcept]
```

Get the hardware offset of the line.

Returns

Offset of the line within the parent chip.

6.23.3.12 operator=() [1/2]

```
GPIOD_CXX_API line_info & gpiod::line_info::operator= (
            const line_info & other ) [noexcept]
```

Copy assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.23.3.13 operator=() [2/2]

```
GPIOD_CXX_API line_info & gpiod::line_info::operator= (
            line_info && other )  [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.23.3.14 used()

```
GPIOD_CXX_API bool gpiod::line_info::used ( ) const  [noexcept]
```

Check if the line is currently in use.

Returns

True if the line is in use, false otherwise.

The user space can't know exactly why a line is busy. It may have been requested by another process or hogged by the kernel. It only matters that the line is used and we can't request it.

The documentation for this class was generated from the following files:

- src/gpiodcxx/line-info.hpp
- src/line-info.cpp

6.24 gpiod::line_request Class Reference

Stores the context of a set of requested GPIO lines.

```
#include <line-request.hpp>
```

Classes

- struct impl

Public Member Functions

- **line_request** (const line_request &other)=delete
- line_request (line_request &&other) noexcept

    Move constructor.
- line_request & **operator=** (const line_request &other)=delete
- line_request & operator= (line_request &&other) noexcept

    Move assignment operator.
- operator bool () const noexcept

    Check if this object is valid.
- void release ()

    Release the requested lines and free all associated resources.
- ::std::string chip_name () const

    Get the name of the chip this request was made on.
- ::std::size_t num_lines () const

    Get the number of requested lines.
- line::offsets offsets () const

    Get the list of offsets of requested lines.
- line::value get_value (line::offset offset)

    Get the value of a single requested line.
- line::values get_values (const line::offsets &offsets)

    Get the values of a subset of requested lines.
- line::values get_values ()

    Get the values of all requested lines.
- void get_values (const line::offsets &offsets, line::values &values)

    Get the values of a subset of requested lines into a vector supplied by the caller.
- void get_values (line::values &values)

    Get the values of all requested lines.
- line_request & set_value (line::offset offset, line::value value)

    Set the value of a single requested line.
- line_request & set_values (const line::value_mappings &values)

    Set the values of a subset of requested lines.
- line_request & set_values (const line::offsets &offsets, const line::values &values)

    Set the values of a subset of requested lines.
- line_request & set_values (const line::values &values)

    Set the values of all requested lines.
- line_request & reconfigure_lines (const line_config &config)

    Apply new config options to requested lines.
- int fd () const

    Get the file descriptor number associated with this line request.
- bool wait_edge_events (const ::std::chrono::nanoseconds &timeout) const

    Wait for edge events on any of the lines requested with edge detection enabled.
- ::std::size_t read_edge_events (edge_event_buffer &buffer)

    Read a number of edge events from this request up to the maximum capacity of the buffer.
- ::std::size_t read_edge_events (edge_event_buffer &buffer, ::std::size_t max_events)

    Read a number of edge events from this request.

6.24.1 Detailed Description

Stores the context of a set of requested GPIO lines.

6.24.2 Constructor & Destructor Documentation

6.24.2.1 line_request()

```
GPIOD_CXX_API gpiod::line_request::line_request (
            line_request && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.24.3 Member Function Documentation

6.24.3.1 chip_name()

```
GPIOD_CXX_API::std::string gpiod::line_request::chip_name ( ) const
```

Get the name of the chip this request was made on.

Returns

Name to the GPIO chip.

6.24.3.2 fd()

```
GPIOD_CXX_API int gpiod::line_request::fd ( ) const
```

Get the file descriptor number associated with this line request.

Returns

File descriptor number.

6.24.3.3 get_value()

```
GPIOD_CXX_API line::value gpiod::line_request::get_value (
            line::offset offset )
```

Get the value of a single requested line.

Parameters

offset	Offset of the line to read within the chip.

Returns

Current line value.

6.24.3.4 get_values() [1/4]

```
GPIOD_CXX_API line::values gpiod::line_request::get_values ( )
```

Get the values of all requested lines.

Returns

List of read values.

6.24.3.5 get_values() [2/4]

```
GPIOD_CXX_API line::values gpiod::line_request::get_values (
            const line::offsets & offsets )
```

Get the values of a subset of requested lines.

Parameters

offsets	Vector of line offsets

Returns

Vector of lines values with indexes of values corresponding to those of the offsets.

6.24.3.6 get_values() [3/4]

```
GPIOD_CXX_API void gpiod::line_request::get_values (
            const line::offsets & offsets,
            line::values & values )
```

Get the values of a subset of requested lines into a vector supplied by the caller.

Parameters

offsets	Vector of line offsets.
values	Vector for storing the values. Its size must be at least that of the offsets vector. The indexes of read values will correspond with those in the offsets vector.

6.24.3.7 get_values() [4/4]

```
GPIOD_CXX_API void gpiod::line_request::get_values (
            line::values & values )
```

Get the values of all requested lines.

Parameters

values	Array in which the values will be stored. Must hold at least the number of elements returned by line_request::num_lines.

6.24.3.8 num_lines()

```
GPIOD_CXX_API::std::size_t gpiod::line_request::num_lines ( ) const
```

Get the number of requested lines.

Returns

Number of lines in this request.

6.24.3.9 offsets()

```
GPIOD_CXX_API line::offsets gpiod::line_request::offsets ( ) const
```

Get the list of offsets of requested lines.

Returns

List of hardware offsets of the lines in this request.

6.24.3.10 operator bool()

```
GPIOD_CXX_API gpiod::line_request::operator bool ( ) const [explicit], [noexcept]
```

Check if this object is valid.

Returns

True if this object's methods can be used, false otherwise. False usually means the request was released. If the user calls any of the methods of this class on an object for which this operator returned false, a logic_error will be thrown.

6.24.3.11 operator=()

```
GPIOD_CXX_API line_request & gpiod::line_request::operator= (
            line_request && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.24.3.12 read_edge_events() [1/2]

```
GPIOD_CXX_API::std::size_t gpiod::line_request::read_edge_events (
            edge_event_buffer & buffer )
```

Read a number of edge events from this request up to the maximum capacity of the buffer.

Parameters

buffer	Edge event buffer to read events into.

Returns

Number of events read.

6.24.3.13 read_edge_events() [2/2]

```
GPIOD_CXX_API::std::size_t gpiod::line_request::read_edge_events (
            edge_event_buffer & buffer,
            ::std::size_t max_events )
```

Read a number of edge events from this request.

Parameters

buffer	Edge event buffer to read events into.
max_events	Maximum number of events to read. Limited by the capacity of the buffer.

Returns

Number of events read.

6.24.3.14 reconfigure_lines()

```
GPIOD_CXX_API line_request & gpiod::line_request::reconfigure_lines (
            const line_config & config )
```

Apply new config options to requested lines.

Parameters

config	New configuration.

Returns

Reference to self.

6.24.3.15 release()

```
GPIOD_CXX_API void gpiod::line_request::release ( )
```

Release the requested lines and free all associated resources.

Note

The object can still be used after this method is called but using any of the mutators will result in throwing a logic_error exception.

6.24.3.16 set_value()

```
line_request & gpiod::line_request::set_value (
            line::offset offset,
            line::value value )
```

Set the value of a single requested line.

Parameters

offset	Offset of the line to set within the chip.
value	New line value.

Returns

Reference to self.

6.24.3.17 set_values() [1/3]

```
GPIOD_CXX_API line_request & gpiod::line_request::set_values (
            const line::offsets & offsets,
            const line::values & values )
```

Set the values of a subset of requested lines.

Parameters

offsets	Vector containing the offsets of lines to set.
values	Vector containing new values with indexes corresponding with those in the offsets vector.

Returns

Reference to self.

6.24.3.18 set_values() [2/3]

```
GPIOD_CXX_API line_request & gpiod::line_request::set_values (
            const line::value_mappings & values )
```

Set the values of a subset of requested lines.

Parameters

values	Vector containing a set of offset->value mappings.

Returns

Reference to self.

6.24.3.19 set_values() [3/3]

```
GPIOD_CXX_API line_request & gpiod::line_request::set_values (
            const line::values & values )
```

Set the values of all requested lines.

Parameters

values	Array of new line values. The size must be equal to the value returned by line_request::num_lines.

Returns

Reference to self.

6.24.3.20 wait_edge_events()

```
GPIOD_CXX_API bool gpiod::line_request::wait_edge_events (
            const ::std::chrono::nanoseconds & timeout ) const
```

Wait for edge events on any of the lines requested with edge detection enabled.

Parameters

timeout	Wait time limit in nanoseconds. If set to 0, the function returns immediately. If set to a negative number, the function blocks indefinitely until an event becomes available.

Returns

True if at least one event is ready to be read. False if the wait timed out.

The documentation for this class was generated from the following files:

- src/gpiodcxx/line-request.hpp
- src/line-request.cpp

6.25 gpiod::line_settings Class Reference

Stores GPIO line settings.

```
#include <line-settings.hpp>
```

Classes

- struct impl

Public Member Functions

- **line_settings** ()

  Initializes the line_settings object with default values.
- line_settings (const line_settings &other)

  Copy constructor.
- line_settings (line_settings &&other) noexcept

  Move constructor.
- line_settings & operator= (const line_settings &other)

  Copy assignment operator.
- line_settings & operator= (line_settings &&other)

  Move assignment operator.
- line_settings & reset () noexcept

  Reset the line settings to default values.
- line_settings & set_direction (line::direction direction)

  Set direction.
- line::direction direction () const

  Get direction.
- line_settings & set_edge_detection (line::edge edge)

  Set edge detection.
- line::edge edge_detection () const

  Get edge detection.
- line_settings & set_bias (line::bias bias)

  Set bias setting.
- line::bias bias () const

  Get bias setting.
- line_settings & set_drive (line::drive drive)

  Set drive setting.
- line::drive drive () const

  Get drive setting.

- line_settings & set_active_low (bool active_low)

  Set the active-low setting.
- bool active_low () const noexcept

  Get the active-low setting.
- line_settings & set_debounce_period (const ::std::chrono::microseconds &period)

  Set debounce period.
- ::std::chrono::microseconds debounce_period () const noexcept

  Get debounce period.
- line_settings & set_event_clock (line::clock event_clock)

  Set the event clock to use for edge event timestamps.
- line::clock event_clock () const

  Get the event clock used for edge event timestamps.
- line_settings & set_output_value (line::value value)

  Set the output value.
- line::value output_value () const

  Get the output value.

6.25.1 Detailed Description

Stores GPIO line settings.

6.25.2 Constructor & Destructor Documentation

6.25.2.1 line_settings() [1/2]

```
GPIOD_CXX_API gpiod::line_settings::line_settings (
            const line_settings & other )
```

Copy constructor.

Parameters

other	Object to copy.

6.25.2.2 line_settings() [2/2]

```
GPIOD_CXX_API gpiod::line_settings::line_settings (
            line_settings && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.25.3 Member Function Documentation

6.25.3.1 active_low()

```
GPIOD_CXX_API bool gpiod::line_settings::active_low ( ) const [noexcept]
```

Get the active-low setting.

Returns

Current active-low setting.

6.25.3.2 bias()

```
GPIOD_CXX_API line::bias gpiod::line_settings::bias ( ) const
```

Get bias setting.

Returns

Current bias.

6.25.3.3 debounce_period()

```
GPIOD_CXX_API::std::chrono::microseconds gpiod::line_settings::debounce_period ( ) const [noexcept]
```

Get debounce period.

Returns

Current debounce period.

6.25.3.4 direction()

```
GPIOD_CXX_API line::direction gpiod::line_settings::direction ( ) const
```

Get direction.

Returns

Current direction setting.

6.25.3.5 drive()

```
GPIOD_CXX_API line::drive gpiod::line_settings::drive ( ) const
```

Get drive setting.

Returns

Current drive.

6.25.3.6 edge_detection()

```
GPIOD_CXX_API line::edge gpiod::line_settings::edge_detection ( ) const
```

Get edge detection.

Returns

Current edge detection setting.

6.25.3.7 event_clock()

```
GPIOD_CXX_API line::clock gpiod::line_settings::event_clock ( ) const
```

Get the event clock used for edge event timestamps.

Returns

Current event clock type.

6.25.3.8 operator=() [1/2]

```
GPIOD_CXX_API line_settings & gpiod::line_settings::operator= (
            const line_settings & other )
```

Copy assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.25.3.9 operator=() [2/2]

```
GPIOD_CXX_API line_settings & gpiod::line_settings::operator= (
            line_settings && other )
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.25.3.10 output_value()

```
GPIOD_CXX_API line::value gpiod::line_settings::output_value ( ) const
```

Get the output value.

Returns

Current output value.

6.25.3.11 reset()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::reset ( )  [noexcept]
```

Reset the line settings to default values.

Returns

Reference to self.

6.25.3.12 set_active_low()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_active_low (
            bool active_low )
```

Set the active-low setting.

Parameters

active_low	New active-low setting.

Returns

Reference to self.

6.25.3.13 set_bias()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_bias (
            line::bias bias )
```

Set bias setting.

Parameters

bias	New bias.

Returns

Reference to self.

6.25.3.14 set_debounce_period()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_debounce_period (
            const ::std::chrono::microseconds & period )
```

Set debounce period.

Parameters

period	New debounce period in microseconds.

Returns

Reference to self.

6.25.3.15 set_direction()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_direction (
            line::direction direction )
```

Set direction.

Parameters

direction	New direction.

Returns

Reference to self.

6.25.3.16 set_drive()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_drive (
            line::drive drive )
```

Set drive setting.

Parameters

drive	New drive.

Returns

Reference to self.

6.25.3.17 set_edge_detection()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_edge_detection (
            line::edge edge )
```

Set edge detection.

Parameters

edge	New edge detection setting.

Returns

Reference to self.

6.25.3.18 set_event_clock()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_event_clock (
            line::clock event_clock )
```

Set the event clock to use for edge event timestamps.

Parameters

event_clock	Clock to use.

Returns

Reference to self.

6.25.3.19 set_output_value()

```
GPIOD_CXX_API line_settings & gpiod::line_settings::set_output_value (
            line::value value )
```

Set the output value.

Parameters

value	New output value.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/line-settings.hpp
- src/line-settings.cpp

6.26 gpiod::line::offset Class Reference

Wrapper around unsigned int for representing line offsets.

`#include <line.hpp>`

Public Member Functions

- offset (unsigned int off=0)

  Constructor with implicit conversion from unsigned int.
- offset (const offset &other)=default

  Copy constructor.
- offset (offset &&other)=default

  Move constructor.
- offset & operator= (const offset &other)=default

  Assignment operator.
- offset & operator= (offset &&other) noexcept=default

  Move assignment operator.
- **operator unsigned int** () const noexcept

  Conversion operator to `unsigned int`.

6.26.1 Detailed Description

Wrapper around unsigned int for representing line offsets.

6.26.2 Constructor & Destructor Documentation

6.26.2.1 offset() [1/3]

```
gpiod::line::offset::offset (
            unsigned int off = 0 )  [inline]
```

Constructor with implicit conversion from unsigned int.

Parameters

off	Line offset.

6.26.2.2 offset() [2/3]

```
gpiod::line::offset::offset (
            const offset & other )   [default]
```

Copy constructor.

Parameters

other	Object to copy.

6.26.2.3 offset() [3/3]

```
gpiod::line::offset::offset (
            offset && other )   [default]
```

Move constructor.

Parameters

other	Object to move.

6.26.3 Member Function Documentation

6.26.3.1 operator=() [1/2]

```
offset & gpiod::line::offset::operator= (
            const offset & other )   [default]
```

Assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.26.3.2 operator=() [2/2]

```
offset & gpiod::line::offset::operator= (
```

`offset && other ) [default], [noexcept]`

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

The documentation for this class was generated from the following file:

- src/gpiodcxx/line.hpp

6.27 gpiod::request_builder Class Reference

Intermediate object storing the configuration for a line request.

```
#include <request-builder.hpp>
```

Classes

- struct impl

Public Member Functions

- **request_builder** (const request_builder &other)=delete
- request_builder (request_builder &&other) noexcept

  Move constructor.
- request_builder & **operator=** (const request_builder &other)=delete
- request_builder & operator= (request_builder &&other) noexcept

  Move assignment operator.
- request_builder & set_request_config (request_config &req_cfg)

  Set the request config for the request.
- const request_config & get_request_config () const noexcept

  Get the current request config.
- request_builder & set_consumer (const ::std::string &consumer) noexcept

  Set consumer in the request config stored by this object.
- request_builder & set_event_buffer_size (::std::size_t event_buffer_size) noexcept

  Set the event buffer size in the request config stored by this object.
- request_builder & set_line_config (line_config &line_cfg)

  Set the line config for this request.
- const line_config & get_line_config () const noexcept

  Get the current line config.
- request_builder & add_line_settings (line::offset offset, const line_settings &settings)

  Add line settings to the line config stored by this object for a single offset.
- request_builder & add_line_settings (const line::offsets &offsets, const line_settings &settings)

  Add line settings to the line config stored by this object for a set of offsets.
- request_builder & set_output_values (const line::values &values)

  Set output values for a number of lines in the line config stored by this object.
- line_request do_request ()

  Make the line request.

6.27.1 Detailed Description

Intermediate object storing the configuration for a line request.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 request_builder()

```
GPIOD_CXX_API gpiod::request_builder::request_builder (
            request_builder && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to be moved.

6.27.3 Member Function Documentation

6.27.3.1 add_line_settings() [1/2]

```
GPIOD_CXX_API request_builder & gpiod::request_builder::add_line_settings (
            const line::offsets & offsets,
            const line_settings & settings )
```

Add line settings to the line config stored by this object for a set of offsets.

Parameters

offsets	Offsets for which to add settings.
settings	Settings to add.

Returns

Reference to self.

6.27.3.2 add_line_settings() [2/2]

```
GPIOD_CXX_API request_builder & gpiod::request_builder::add_line_settings (
            line::offset offset,
            const line_settings & settings )
```

Add line settings to the line config stored by this object for a single offset.

Parameters

offset	Offset for which to add settings.
settings	Line settings to use.

Returns

Reference to self.

6.27.3.3 do_request()

```
GPIOD_CXX_API line_request gpiod::request_builder::do_request ( )
```

Make the line request.

Returns

New line_request object.

6.27.3.4 get_line_config()

```
GPIOD_CXX_API const line_config & gpiod::request_builder::get_line_config ( ) const [noexcept]
```

Get the current line config.

Returns

Const reference to the current line config stored by this object.

6.27.3.5 get_request_config()

```
GPIOD_CXX_API const request_config & gpiod::request_builder::get_request_config ( ) const
[noexcept]
```

Get the current request config.

Returns

Const reference to the current request config stored by this object.

6.27.3.6 operator=()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::operator= (
            request_builder && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to be moved.

Returns

Reference to self.

6.27.3.7 set_consumer()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::set_consumer (
            const ::std::string & consumer ) [noexcept]
```

Set consumer in the request config stored by this object.

Parameters

consumer	New consumer string.

Returns

Reference to self.

6.27.3.8 set_event_buffer_size()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::set_event_buffer_size (
            ::std::size_t event_buffer_size ) [noexcept]
```

Set the event buffer size in the request config stored by this object.

Parameters

event_buffer_size	New event buffer size.

Returns

Reference to self.

6.27.3.9 set_line_config()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::set_line_config (
            line_config & line_cfg )
```

Set the line config for this request.

Parameters

line_cfg	Line config to use.

Returns

Reference to self.

6.27.3.10 set_output_values()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::set_output_values (
            const line::values & values )
```

Set output values for a number of lines in the line config stored by this object.

Parameters

values	Buffer containing the output values.

Returns

Reference to self.

6.27.3.11 set_request_config()

```
GPIOD_CXX_API request_builder & gpiod::request_builder::set_request_config (
            request_config & req_cfg )
```

Set the request config for the request.

Parameters

req_cfg	Request config to use.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/request-builder.hpp
- src/request-builder.cpp

6.28 gpiod::request_config Class Reference

Stores a set of options passed to the kernel when making a line request.

```
#include <request-config.hpp>
```

Classes

- struct impl

Public Member Functions

- **request_config** ()
  Constructor.
- **request_config** (const request_config &other)=delete
- request_config (request_config &&other) noexcept
  Move constructor.
- request_config & operator= (request_config &&other) noexcept
  Move assignment operator.
- request_config & set_consumer (const ::std::string &consumer) noexcept
  Set the consumer name.
- ::std::string consumer () const noexcept
  Get the consumer name.
- request_config & set_event_buffer_size (::std::size_t event_buffer_size) noexcept
  Set the size of the kernel event buffer.
- ::std::size_t event_buffer_size () const noexcept
  Get the edge event buffer size from this request config.

6.28.1 Detailed Description

Stores a set of options passed to the kernel when making a line request.

6.28.2 Constructor & Destructor Documentation

6.28.2.1 request_config()

```
GPIOD_CXX_API gpiod::request_config::request_config (
            request_config && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.28.3 Member Function Documentation

6.28.3.1 consumer()

```
GPIOD_CXX_API::std::string gpiod::request_config::consumer ( ) const  [noexcept]
```

Get the consumer name.

Returns

Currently configured consumer name. May be an empty string.

6.28.3.2 event_buffer_size()

```
GPIOD_CXX_API::std::size_t gpiod::request_config::event_buffer_size ( ) const [noexcept]
```

Get the edge event buffer size from this request config.

Returns

Current edge event buffer size setting.

6.28.3.3 operator=()

```
GPIOD_CXX_API request_config & gpiod::request_config::operator= (
            request_config && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.28.3.4 set_consumer()

```
GPIOD_CXX_API request_config & gpiod::request_config::set_consumer (
            const ::std::string & consumer ) [noexcept]
```

Set the consumer name.

Parameters

consumer	New consumer name.

Returns

Reference to self.

6.28.3.5 set_event_buffer_size()

```
GPIOD_CXX_API request_config & gpiod::request_config::set_event_buffer_size (
            ::std::size_t event_buffer_size ) [noexcept]
```

Set the size of the kernel event buffer.

Parameters

event_buffer_size	New event buffer size.

Returns

Reference to self.

Note

The kernel may adjust the value if it's too high. If set to 0, the default value will be used.

The documentation for this class was generated from the following files:

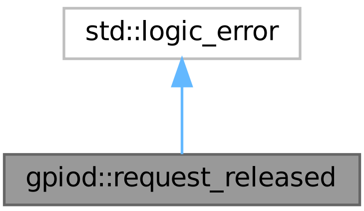
- src/gpiodcxx/request-config.hpp
- src/request-config.cpp

6.29 gpiod::request_released Class Reference

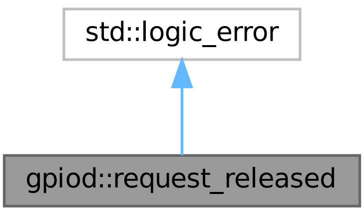
Exception thrown when an already released line request is used.

`#include <exception.hpp>`

Inheritance diagram for gpiod::request_released:



Collaboration diagram for gpiod::request_released:



Public Member Functions

- request_released (const ::std::string &what)
  Constructor.
- request_released (const request_released &other) noexcept
  Copy constructor.
- request_released (request_released &&other) noexcept
  Move constructor.
- request_released & operator= (const request_released &other) noexcept
  Assignment operator.
- request_released & operator= (request_released &&other) noexcept
  Move assignment operator.

6.29.1 Detailed Description

Exception thrown when an already released line request is used.

6.29.2 Constructor & Destructor Documentation

6.29.2.1 request_released() [1/3]

```
GPIOD_CXX_API gpiod::request_released::request_released (
            const ::std::string & what )  [explicit]
```

Constructor.

Parameters

what	Human readable reason for error.

6.29.2.2 request_released() [2/3]

```
GPIOD_CXX_API gpiod::request_released::request_released (
            const request_released & other )  [noexcept]
```

Copy constructor.

Parameters

other	Object to copy from.

6.29.2.3 request_released() [3/3]

```
GPIOD_CXX_API gpiod::request_released::request_released (
            request_released && other )  [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.29.3 Member Function Documentation

6.29.3.1 operator=() [1/2]

```
GPIOD_CXX_API request_released & gpiod::request_released::operator= (
            const request_released & other ) [noexcept]
```

Assignment operator.

Parameters

other	Object to copy from.

Returns

Reference to self.

6.29.3.2 operator=() [2/2]

```
GPIOD_CXX_API request_released & gpiod::request_released::operator= (
            request_released && other ) [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

The documentation for this class was generated from the following files:

- src/gpiodcxx/exception.hpp
- src/exception.cpp

6.30 gpiod::timestamp Class Reference

Stores the edge and info event timestamps as returned by the kernel and allows to convert them to std::chrono↩ ::time_point.

```
#include <timestamp.hpp>
```

Public Types

- using **time_point_monotonic** = ::std::chrono::time_point< ::std::chrono::steady_clock >

  Monotonic time_point.
- using **time_point_realtime** = ::std::chrono::time_point< ::std::chrono::system_clock, ::std::chrono::nanoseconds >

  Real-time time_point.

Public Member Functions

- timestamp (::std::uint64_t ns)

  Constructor with implicit conversion from `uint64_t`.
- timestamp (const timestamp &other) noexcept=default

  Copy constructor.
- timestamp (timestamp &&other) noexcept=default

  Move constructor.
- timestamp & operator= (const timestamp &other) noexcept=default

  Assignment operator.
- timestamp & operator= (timestamp &&other) noexcept=default

  Move assignment operator.
- **operator::std::uint64_t** () noexcept

  Conversion operator to `std::uint64_t`.
- ::std::uint64_t ns () const noexcept

  Get the timestamp in nanoseconds.
- time_point_monotonic to_time_point_monotonic () const

  Convert the timestamp to a monotonic time_point.
- time_point_realtime to_time_point_realtime () const

  Convert the timestamp to a real-time time_point.

6.30.1 Detailed Description

Stores the edge and info event timestamps as returned by the kernel and allows to convert them to std::chrono::time_point.

6.30.2 Constructor & Destructor Documentation

6.30.2.1 timestamp() [1/3]

```
gpiod::timestamp::timestamp (
            ::std::uint64_t ns )  [inline]
```

Constructor with implicit conversion from `uint64_t`.

Parameters

ns	Timestamp in nanoseconds.

6.30.2.2 timestamp() [2/3]

```
gpiod::timestamp::timestamp (
            const timestamp & other )  [default], [noexcept]
```

Copy constructor.

Parameters

other	Object to copy.

6.30.2.3 timestamp() [3/3]

```
gpiod::timestamp::timestamp (
            timestamp && other )  [default], [noexcept]
```

Move constructor.

Parameters

other	Object to move.

6.30.3 Member Function Documentation

6.30.3.1 ns()

```
::std::uint64_t gpiod::timestamp::ns ( ) const  [inline], [noexcept]
```

Get the timestamp in nanoseconds.

Returns

Timestamp in nanoseconds.

6.30.3.2 operator=() [1/2]

```
timestamp & gpiod::timestamp::operator= (
            const timestamp & other )  [default], [noexcept]
```

Assignment operator.

Parameters

other	Object to copy.

Returns

Reference to self.

6.30.3.3 operator=() [2/2]

```
timestamp & gpiod::timestamp::operator= (
            timestamp && other ) [default], [noexcept]
```

Move assignment operator.

Parameters

other	Object to move.

Returns

Reference to self.

6.30.3.4 to_time_point_monotonic()

```
time_point_monotonic gpiod::timestamp::to_time_point_monotonic ( ) const [inline]
```

Convert the timestamp to a monotonic time_point.

Returns

time_point associated with the steady clock.

6.30.3.5 to_time_point_realtime()

```
time_point_realtime gpiod::timestamp::to_time_point_realtime ( ) const [inline]
```

Convert the timestamp to a real-time time_point.

Returns

time_point associated with the system clock.

The documentation for this class was generated from the following file:

- src/gpiodcxx/timestamp.hpp

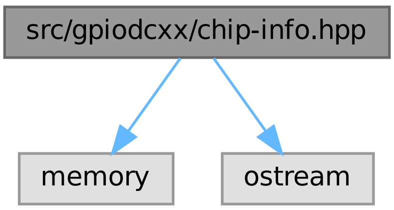
Chapter 7

File Documentation

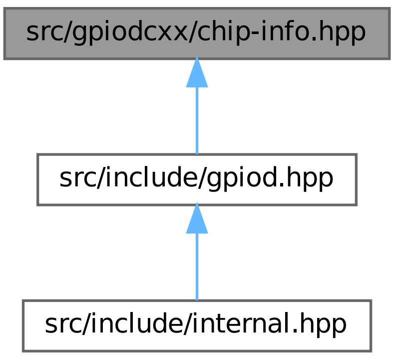
7.1 src/gpiodcxx/chip-info.hpp File Reference

```
#include <memory>
#include <ostream>
```

Include dependency graph for chip-info.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::chip_info

    Represents an immutable snapshot of GPIO chip information.

Functions

- GPIOD_CXX_API::std::ostream & gpiod::operator<< (::std::ostream &out, const chip_info &chip)

    Stream insertion operator for GPIO chip objects.

7.1.1 Function Documentation

7.1.1.1 operator<<()

```
std::ostream & gpiod::operator<< (
              ::std::ostream & out,
              const chip_info & chip )
```

Stream insertion operator for GPIO chip objects.

Parameters

out	Output stream to write to.
chip	GPIO chip to insert into the output stream.

Returns

Reference to out.

7.2 chip-info.hpp

Go to the documentation of this file.

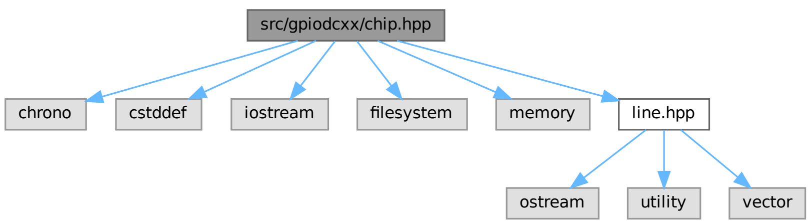
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_CHIP_INFO_HPP__
#define __LIBGPIOD_CXX_CHIP_INFO_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <memory>
#include <ostream>

namespace gpiod {

class chip;

class chip_info final
{
public:

    chip_info(const chip_info& other);

    chip_info(chip_info&& other) noexcept;

    ~chip_info();
```

```
    chip_info& operator=(const chip_info& other);
    chip_info& operator=(chip_info&& other) noexcept;
    ::std::string name() const noexcept;
    ::std::string label() const noexcept;
    ::std::size_t num_lines() const noexcept;
private:
    chip_info();
    struct impl;
    ::std::shared_ptr<impl> _m_priv;
    friend chip;
};
::std::ostream& operator«(::std::ostream& out, const chip_info& chip);
} /* namespace gpiod */
#endif /* __LIBGPIOD_CXX_CHIP_INFO_HPP__ */
```

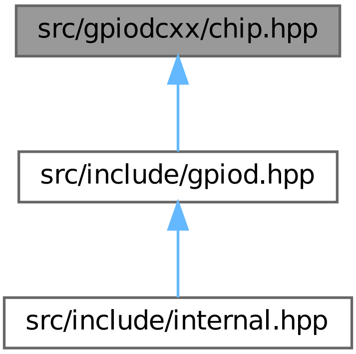
7.3 src/gpiodcxx/chip.hpp File Reference

```
#include <chrono>
#include <cstddef>
#include <iostream>
#include <filesystem>
#include <memory>
#include "line.hpp"
```

Include dependency graph for chip.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::chip

  Represents a GPIO chip.

Functions

- GPIOD_CXX_API::std::ostream & gpiod::operator<< (::std::ostream &out, const chip &chip)

  Stream insertion operator for GPIO chip objects.

7.3.1 Function Documentation

7.3.1.1 operator<<()

```
std::ostream & gpiod::operator<< (
             ::std::ostream & out,
             const chip & chip )
```

Stream insertion operator for GPIO chip objects.

Parameters

out	Output stream to write to.
chip	GPIO chip to insert into the output stream.

Returns

Reference to out.

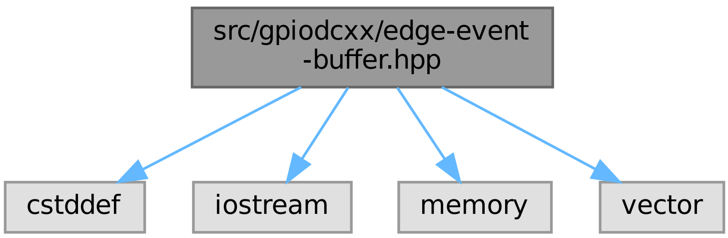
7.4 chip.hpp

Go to the documentation of this file.

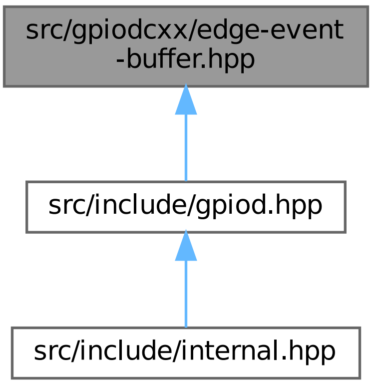
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_CHIP_HPP__
#define __LIBGPIOD_CXX_CHIP_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <chrono>
#include <cstddef>
#include <iostream>
#include <filesystem>
#include <memory>

#include "line.hpp"

namespace gpiod {

class chip_info;
class info_event;
class line_config;
class line_info;
class line_request;
class request_builder;
class request_config;

class chip final
{
public:

    explicit chip(const ::std::filesystem::path& path);

    chip(chip&& other) noexcept;

    ~chip();

    chip& operator=(const chip& other) = delete;

    chip& operator=(chip&& other) noexcept;

    explicit operator bool() const noexcept;

    void close();

    ::std::filesystem::path path() const;

    chip_info get_info() const;

    line_info get_line_info(line::offset offset) const;

    line_info watch_line_info(line::offset offset) const;

    void unwatch_line_info(line::offset offset) const;

    int fd() const;

    bool wait_info_event(const ::std::chrono::nanoseconds& timeout) const;

    info_event read_info_event() const;

    int get_line_offset_from_name(const ::std::string& name) const;

    request_builder prepare_request();

private:

    struct impl;

    ::std::shared_ptr<impl> _m_priv;

    chip(const chip& other);

    friend request_builder;
};

::std::ostream& operator«(::std::ostream& out, const chip& chip);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_CHIP_HPP__ */
```

7.5 src/gpiodcxx/edge-event-buffer.hpp File Reference

```
#include <cstddef>
#include <iostream>
#include <memory>
#include <vector>
```
Include dependency graph for edge-event-buffer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::edge_event_buffer

    Object into which edge events are read for better performance.

Functions

- GPIOD_CXX_API::std::ostream & gpiod::operator<< (::std::ostream &out, const edge_event_buffer &buf)

    Stream insertion operator for GPIO edge event buffer objects.

7.5.1 Function Documentation

7.5.1.1 operator<<()

```
std::ostream & gpiod::operator<< (
              ::std::ostream & out,
              const edge_event_buffer & buf )
```

Stream insertion operator for GPIO edge event buffer objects.

Parameters

out	Output stream to write to.
buf	GPIO edge event buffer object to insert into the output stream.

Returns

Reference to out.

7.6 edge-event-buffer.hpp

Go to the documentation of this file.

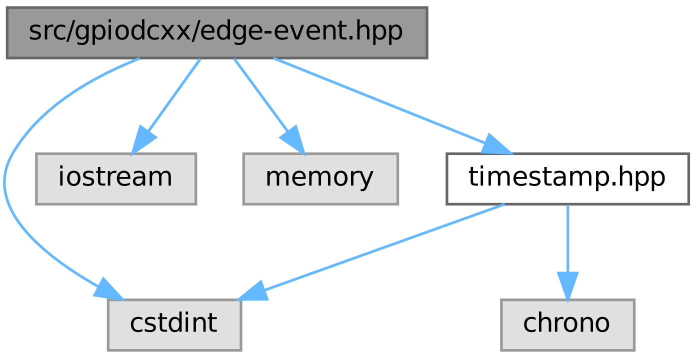
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_EDGE_EVENT_BUFFER_HPP__
#define __LIBGPIOD_CXX_EDGE_EVENT_BUFFER_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <cstddef>
#include <iostream>
#include <memory>
#include <vector>

namespace gpiod {

class edge_event;
class line_request;

class edge_event_buffer final
{
public:

    using const_iterator = ::std::vector<edge_event>::const_iterator;

    explicit edge_event_buffer(::std::size_t capacity = 64);

    edge_event_buffer(const edge_event_buffer& other) = delete;

    edge_event_buffer(edge_event_buffer&& other) noexcept;

    ~edge_event_buffer();

    edge_event_buffer& operator=(const edge_event_buffer& other) = delete;

    edge_event_buffer& operator=(edge_event_buffer&& other) noexcept;

    const edge_event& get_event(unsigned int index) const;

    ::std::size_t num_events() const;

    ::std::size_t capacity() const noexcept;

    const_iterator begin() const noexcept;

```

```
    const_iterator end() const noexcept;

private:

    struct impl;

    ::std::unique_ptr<impl> _m_priv;

    friend line_request;
};

::std::ostream& operator«(::std::ostream& out, const edge_event_buffer& buf);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_EDGE_EVENT_BUFFER_HPP__ */
```

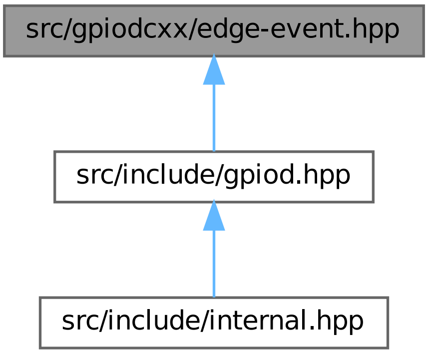
7.7 src/gpiodcxx/edge-event.hpp File Reference

```
#include <cstdint>
#include <iostream>
#include <memory>
#include "timestamp.hpp"
```

Include dependency graph for edge-event.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::edge_event

  Immutable object containing data about a single edge event.

Functions

- GPIOD_CXX_API::std::ostream & gpiod::operator<< (::std::ostream &out, const edge_event &event)

  Stream insertion operator for edge events.

7.7.1 Function Documentation

7.7.1.1 operator<<()

```
std::ostream & gpiod::operator<< (
            ::std::ostream & out,
            const edge_event & event )
```

Stream insertion operator for edge events.

Parameters

out	Output stream to write to.
event	Edge event to insert into the output stream.

Returns

Reference to out.

7.8 edge-event.hpp

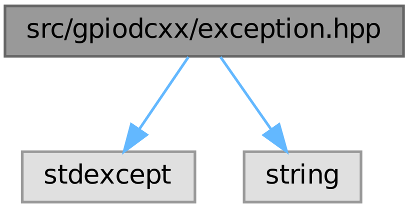
Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_EDGE_EVENT_HPP__
#define __LIBGPIOD_CXX_EDGE_EVENT_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <cstdint>
#include <iostream>
#include <memory>

#include "timestamp.hpp"

namespace gpiod {

class edge_event_buffer;

class edge_event final
{
public:

    enum class event_type
    {
        RISING_EDGE = 1,
        FALLING_EDGE,
    };

    edge_event(const edge_event& other);

    edge_event(edge_event&& other) noexcept;

    ~edge_event();

    edge_event& operator=(const edge_event& other);

    edge_event& operator=(edge_event&& other) noexcept;

    event_type type() const;

    timestamp timestamp_ns() const noexcept;

    line::offset line_offset() const noexcept;

    unsigned long global_seqno() const noexcept;

    unsigned long line_seqno() const noexcept;

private:

    edge_event();

    struct impl;
    struct impl_managed;
    struct impl_external;

    ::std::shared_ptr<impl> _m_priv;

    friend edge_event_buffer;
};

::std::ostream& operator«(::std::ostream& out, const edge_event& event);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_EDGE_EVENT_HPP__ */
```

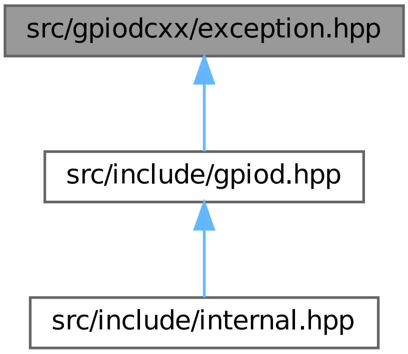
7.9 src/gpiodcxx/exception.hpp File Reference

```
#include <stdexcept>
#include <string>
```

Include dependency graph for exception.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::chip_closed

    Exception thrown when an already closed chip is used.
- class gpiod::request_released

    Exception thrown when an already released line request is used.
- class gpiod::bad_mapping

    Exception thrown when the core C library returns an invalid value for any of the line_info properties.

7.10 exception.hpp

Go to the documentation of this file.

```
00001 /* SPDX-License-Identifier: LGPL-2.1-or-later */
00002 /* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */
00003
00008 #ifndef __LIBGPIOD_CXX_EXCEPTION_HPP__
```

```
#define __LIBGPIOD_CXX_EXCEPTION_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <stdexcept>
#include <string>

namespace gpiod {

class GPIOD_CXX_API chip_closed final : public ::std::logic_error
{
public:

    explicit chip_closed(const ::std::string& what);

    chip_closed(const chip_closed& other) noexcept;

    chip_closed(chip_closed&& other) noexcept;

    chip_closed& operator=(const chip_closed& other) noexcept;

    chip_closed& operator=(chip_closed&& other) noexcept;

    ~chip_closed();
};

class GPIOD_CXX_API request_released final : public ::std::logic_error
{
public:

    explicit request_released(const ::std::string& what);

    request_released(const request_released& other) noexcept;

    request_released(request_released&& other) noexcept;

    request_released& operator=(const request_released& other) noexcept;

    request_released& operator=(request_released&& other) noexcept;

    ~request_released();
};

class GPIOD_CXX_API bad_mapping final : public ::std::runtime_error
{
public:

    explicit bad_mapping(const ::std::string& what);

    bad_mapping(const bad_mapping& other) noexcept;

    bad_mapping(bad_mapping&& other) noexcept;

    bad_mapping& operator=(const bad_mapping& other) noexcept;

    bad_mapping& operator=(bad_mapping&& other) noexcept;

    ~bad_mapping();
};

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_EXCEPTION_HPP__ */
```

7.11 info-event.hpp

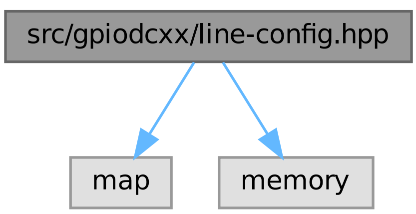
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_INFO_EVENT_HPP__
#define __LIBGPIOD_CXX_INFO_EVENT_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <cstdint>
#include <iostream>
#include <memory>

#include "timestamp.hpp"
```

```
namespace gpiod {
class chip;
class line_info;
class info_event final
{
public:
    enum class event_type
    {
        LINE_REQUESTED = 1,
        LINE_RELEASED,
        LINE_CONFIG_CHANGED,
    };
    info_event(const info_event& other);
    info_event(info_event&& other) noexcept;
    ~info_event();
    info_event& operator=(const info_event& other);
    info_event& operator=(info_event&& other) noexcept;
    event_type type() const;
    ::std::uint64_t timestamp_ns() const noexcept;
    const line_info& get_line_info() const noexcept;
private:
    info_event();
    struct impl;
    ::std::shared_ptr<impl> _m_priv;
    friend chip;
};
::std::ostream& operator«(::std::ostream& out, const info_event& event);
} /* namespace gpiod */
#endif /* __LIBGPIOD_CXX_INFO_EVENT_HPP__ */
```

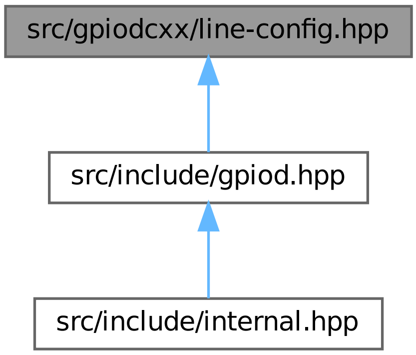
7.12 src/gpiodcxx/line-config.hpp File Reference

`#include <map>`
`#include <memory>`

Include dependency graph for line-config.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::line_config

  Contains a set of line config options used in line requests and reconfiguration.

Functions

- ::std::ostream & gpiod::operator<< (::std::ostream &out, const line_config &config)

  Stream insertion operator for GPIO line config objects.

7.12.1 Function Documentation

7.12.1.1 operator<<()

```
GPIOD_CXX_API::std::ostream & gpiod::operator<< (
             ::std::ostream & out,
             const line_config & config )
```

Stream insertion operator for GPIO line config objects.

Parameters

out	Output stream to write to.
config	Line config object to insert into the output stream.

Returns

Reference to out.

7.13 line-config.hpp

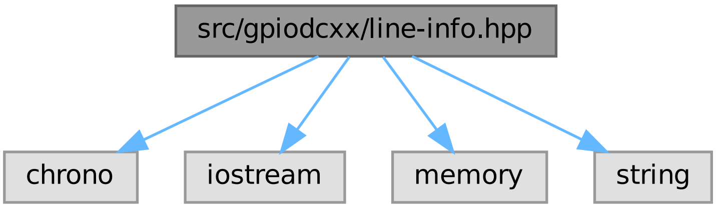
Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_LINE_CONFIG_HPP__
#define __LIBGPIOD_CXX_LINE_CONFIG_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <map>
#include <memory>

namespace gpiod {

class chip;
class line_request;
class line_settings;

class line_config final
{
public:


    line_config();

    line_config(const line_config& other) = delete;

    line_config(line_config&& other) noexcept;

    ~line_config();

    line_config& operator=(line_config&& other) noexcept;

    line_config& reset() noexcept;

    line_config& add_line_settings(line::offset offset, const line_settings& settings);

    line_config& add_line_settings(const line::offsets& offsets, const line_settings& settings);

    line_config& set_output_values(const line::values& values);

    ::std::map<line::offset, line_settings> get_line_settings() const;

private:

    struct impl;

    ::std::shared_ptr<impl> _m_priv;

    line_config& operator=(const line_config& other);

    friend line_request;
    friend request_builder;
};

::std::ostream& operator«(::std::ostream& out, const line_config& config);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_LINE_CONFIG_HPP__ */
```

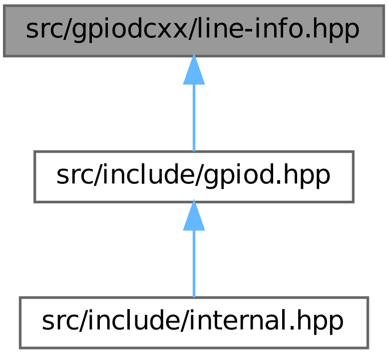
7.14 src/gpiodcxx/line-info.hpp File Reference

```
#include <chrono>
#include <iostream>
#include <memory>
#include <string>
```

Include dependency graph for line-info.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::line_info

    Contains an immutable snapshot of the line's state at the time when the object of this class was instantiated.

Functions

- ::std::ostream & gpiod::operator<< (::std::ostream &out, const line_info &info)

    Stream insertion operator for GPIO line info objects.

7.14.1 Function Documentation

7.14.1.1 operator<<()

```
GPIOD_CXX_API::std::ostream & gpiod::operator<< (
             ::std::ostream & out,
             const line_info & info )
```

Stream insertion operator for GPIO line info objects.

Parameters

out	Output stream to write to.
info	GPIO line info object to insert into the output stream.

Returns

Reference to out.

7.15 line-info.hpp

Go to the documentation of this file.

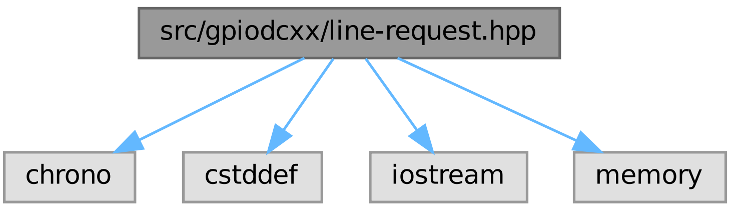
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_LINE_INFO_HPP__
#define __LIBGPIOD_CXX_LINE_INFO_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <chrono>
#include <iostream>
#include <memory>
#include <string>

namespace gpiod {

class chip;
class info_event;

class line_info final
{
public:

    line_info(const line_info& other) noexcept;

    line_info(line_info&& other) noexcept;

    ~line_info();

    line_info& operator=(const line_info& other) noexcept;

    line_info& operator=(line_info&& other) noexcept;

    line::offset offset() const noexcept;

    ::std::string name() const noexcept;

    bool used() const noexcept;

    ::std::string consumer() const noexcept;

    line::direction direction() const;

    line::edge edge_detection() const;

    line::bias bias() const;

    line::drive drive() const;

    bool active_low() const noexcept;

    bool debounced() const noexcept;

    ::std::chrono::microseconds debounce_period() const noexcept;

    line::clock event_clock() const;

private:

    line_info();

    struct impl;

```

```
    ::std::shared_ptr<impl> _m_priv;

    friend chip;
    friend info_event;
};

::std::ostream& operator«(::std::ostream& out, const line_info& info);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_LINE_INFO_HPP__ */
```

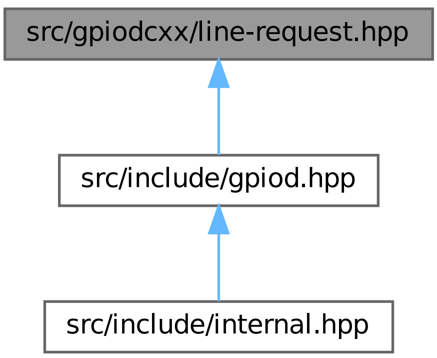
7.16 src/gpiodcxx/line-request.hpp File Reference

```
#include <chrono>
#include <cstddef>
#include <iostream>
#include <memory>
```

Include dependency graph for line-request.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::line_request

    Stores the context of a set of requested GPIO lines.

Functions

- ::std::ostream & gpiod::operator<< (::std::ostream &out, const line_request &request)
  Stream insertion operator for line requests.

7.16.1 Function Documentation

7.16.1.1 operator<<()

```
GPIOD_CXX_API::std::ostream & gpiod::operator<< (
            ::std::ostream & out,
            const line_request & request )
```

Stream insertion operator for line requests.

Parameters

out	Output stream to write to.
request	Line request object to insert into the output stream.

Returns

Reference to out.

7.17 line-request.hpp

Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_LINE_REQUEST_HPP__
#define __LIBGPIOD_CXX_LINE_REQUEST_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <chrono>
#include <cstddef>
#include <iostream>
#include <memory>

namespace gpiod {

class chip;
class edge_event;
class edge_event_buffer;
class line_config;

class line_request final
{
public:

    line_request(const line_request& other) = delete;

    line_request(line_request&& other) noexcept;

    ~line_request();

    line_request& operator=(const line_request& other) = delete;

    line_request& operator=(line_request&& other) noexcept;

```

```
    explicit operator bool() const noexcept;

    void release();

    ::std::string chip_name() const;

    ::std::size_t num_lines() const;

    line::offsets offsets() const;

    line::value get_value(line::offset offset);

    line::values get_values(const line::offsets& offsets);

    line::values get_values();

    void get_values(const line::offsets& offsets, line::values& values);

    void get_values(line::values& values);

    line_request& set_value(line::offset offset, line::value value);

    line_request& set_values(const line::value_mappings& values);

    line_request& set_values(const line::offsets& offsets, const line::values& values);

    line_request& set_values(const line::values& values);

    line_request& reconfigure_lines(const line_config& config);

    int fd() const;

    bool wait_edge_events(const ::std::chrono::nanoseconds& timeout) const;

    ::std::size_t read_edge_events(edge_event_buffer& buffer);

    ::std::size_t read_edge_events(edge_event_buffer& buffer, ::std::size_t max_events);

private:

    line_request();

    struct impl;

    ::std::unique_ptr<impl> _m_priv;

    friend request_builder;
};

::std::ostream& operator«(::std::ostream& out, const line_request& request);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_LINE_REQUEST_HPP__ */
```

7.18 line-settings.hpp

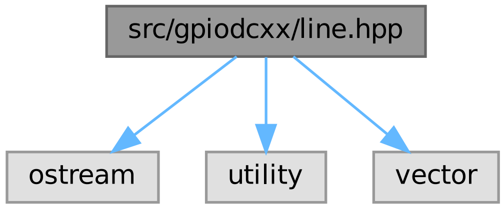
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_LINE_SETTINGS_HPP__
#define __LIBGPIOD_CXX_LINE_SETTINGS_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <chrono>
#include <memory>

#include "line.hpp"

namespace gpiod {

class line_config;

class line_settings final
{
public:

    line_settings();

    line_settings(const line_settings& other);
```

```
    line_settings(line_settings&& other) noexcept;
    ~line_settings();
    line_settings& operator=(const line_settings& other);
    line_settings& operator=(line_settings&& other);
    line_settings& reset() noexcept;
    line_settings& set_direction(line::direction direction);
    line::direction direction() const;
    line_settings& set_edge_detection(line::edge edge);
    line::edge edge_detection() const;
    line_settings& set_bias(line::bias bias);
    line::bias bias() const;
    line_settings& set_drive(line::drive drive);
    line::drive drive() const;
    line_settings& set_active_low(bool active_low);
    bool active_low() const noexcept;
    line_settings& set_debounce_period(const ::std::chrono::microseconds& period);
    ::std::chrono::microseconds debounce_period() const noexcept;
    line_settings& set_event_clock(line::clock event_clock);
    line::clock event_clock() const;
    line_settings& set_output_value(line::value value);
    line::value output_value() const;
private:
    struct impl;
    ::std::unique_ptr<impl> _m_priv;
    friend line_config;
};
::std::ostream& operator«(::std::ostream& out, const line_settings& settings);
} /* namespace gpiod */
#endif /* __LIBGPIOD_CXX_LINE_SETTINGS_HPP__ */
```

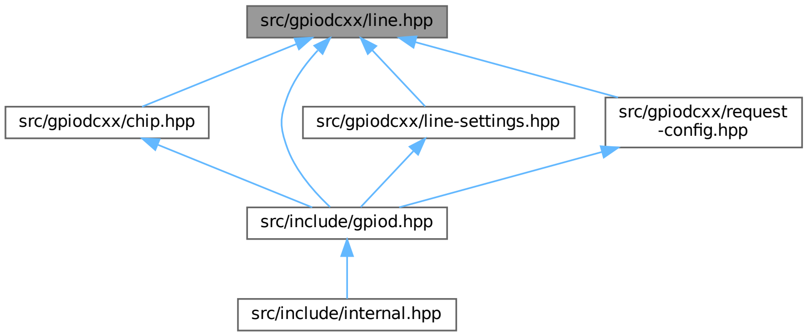
7.19 src/gpiodcxx/line.hpp File Reference

```
#include <ostream>
#include <utility>
#include <vector>
```

Include dependency graph for line.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::line::offset

    Wrapper around unsigned int for representing line offsets.

Namespaces

- namespace gpiod::line

    Namespace containing various type definitions for GPIO lines.

Typedefs

- using **gpiod::line::offsets** = ::std::vector< offset >

    Vector of line offsets.
- using **gpiod::line::values** = ::std::vector< value >

    Vector of line values.
- using **gpiod::line::value_mapping** = ::std::pair< offset, value >

    Represents a mapping of a line offset to line logical state.
- using **gpiod::line::value_mappings** = ::std::vector< value_mapping >

    Vector of offset->value mappings. Each mapping is defined as a pair of an unsigned and signed integers.

Enumerations

- enum class gpiod::line::value { gpiod::line::INACTIVE = 0 , gpiod::line::ACTIVE = 1 }

  Logical line states.
- enum class gpiod::line::direction { gpiod::line::AS_IS = 1 , gpiod::line::INPUT , gpiod::line::OUTPUT }

  Direction settings.
- enum class gpiod::line::edge { gpiod::line::NONE = 1 , gpiod::line::RISING , gpiod::line::FALLING , gpiod::line::BOTH }

  Edge detection settings.
- enum class gpiod::line::bias {
  gpiod::line::AS_IS = 1 , gpiod::line::UNKNOWN , gpiod::line::DISABLED , gpiod::line::PULL_UP ,
  gpiod::line::PULL_DOWN }

  Internal bias settings.
- enum class gpiod::line::drive { gpiod::line::PUSH_PULL = 1 , gpiod::line::OPEN_DRAIN , gpiod::line::OPEN_SOURCE }

  Drive settings.
- enum class gpiod::line::clock { gpiod::line::MONOTONIC = 1 , gpiod::line::REALTIME , **HTE** }

  Event clock settings.

Functions

- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, value val)

  Stream insertion operator for logical line values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, direction dir)

  Stream insertion operator for direction values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, edge edge)

  Stream insertion operator for edge detection values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, bias bias)

  Stream insertion operator for bias values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, drive drive)

  Stream insertion operator for drive values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, clock clock)

  Stream insertion operator for event clock values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, const values &vals)

  Stream insertion operator for the list of output values.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, const offsets &offs)

  Stream insertion operator for the list of line offsets.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, const value_mapping &mapping)

  Stream insertion operator for the offset-to-value mapping.
- ::std::ostream & gpiod::line::operator<< (::std::ostream &out, const value_mappings &mappings)

  Stream insertion operator for the list of offset-to-value mappings.

7.20 line.hpp

Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_LINE_HPP__
#define __LIBGPIOD_CXX_LINE_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
```

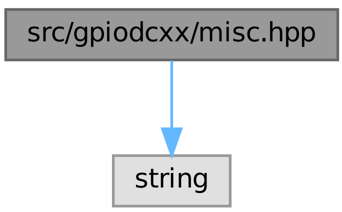
```
#error "Only gpiod.hpp can be included directly."
#endif

#include <ostream>
#include <utility>
#include <vector>

namespace gpiod {

namespace line {

class offset
{
public:
    offset(unsigned int off = 0) : _m_offset(off) {    }

    offset(const offset& other) = default;

    offset(offset&& other) = default;

    ~offset() = default;

    offset& operator=(const offset& other) = default;

    offset& operator=(offset&& other) noexcept = default;

    operator unsigned int() const noexcept
    {
        return this->_m_offset;
    }

private:
    unsigned int _m_offset;
};

enum class value
{
    INACTIVE = 0,
    ACTIVE = 1,
};

enum class direction
{
    AS_IS = 1,
    INPUT,
    OUTPUT,
};

enum class edge
{
    NONE = 1,
    RISING,
    FALLING,
    BOTH,
};

enum class bias
{
    AS_IS = 1,
    UNKNOWN,
    DISABLED,
    PULL_UP,
    PULL_DOWN,
};

enum class drive
{
    PUSH_PULL = 1,
    OPEN_DRAIN,
    OPEN_SOURCE,
};

enum class clock
{
    MONOTONIC = 1,
    REALTIME,
    HTE,
    /*« Line uses the hardware timestamp engine for event timestamps. */
};

using offsets = ::std::vector<offset>;

using values = ::std::vector<value>;

using value_mapping = ::std::pair<offset, value>;

using value_mappings = ::std::vector<value_mapping>;
```

```

::std::ostream& operator«(::std::ostream& out, value val);

::std::ostream& operator«(::std::ostream& out, direction dir);

::std::ostream& operator«(::std::ostream& out, edge edge);

::std::ostream& operator«(::std::ostream& out, bias bias);

::std::ostream& operator«(::std::ostream& out, drive drive);

::std::ostream& operator«(::std::ostream& out, clock clock);

::std::ostream& operator«(::std::ostream& out, const values& vals);

::std::ostream& operator«(::std::ostream& out, const offsets& offs);

::std::ostream& operator«(::std::ostream& out, const value_mapping& mapping);

::std::ostream& operator«(::std::ostream& out, const value_mappings& mappings);

} /* namespace line */

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_LINE_HPP__ */
```

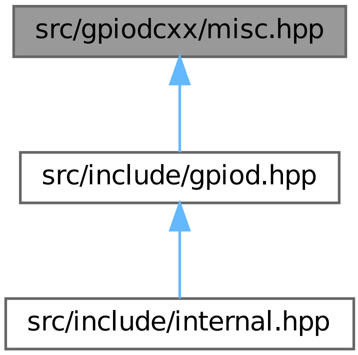
7.21 src/gpiodcxx/misc.hpp File Reference

`#include <string>`

Include dependency graph for misc.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- bool gpiod::is_gpiochip_device (const ::std::filesystem::path &path)

  Check if the file pointed to by path is a GPIO chip character device.
- const ::std::string & gpiod::api_version ()

  Get the human readable version string for libgpiod API.

7.21.1 Function Documentation

7.21.1.1 api_version()

```
GPIOD_CXX_APIconst::std::string & gpiod::api_version ( )
```

Get the human readable version string for libgpiod API.

Returns

String containing the library version.

7.21.1.2 is_gpiochip_device()

```
GPIOD_CXX_API bool gpiod::is_gpiochip_device (
             const ::std::filesystem::path & path )
```

Check if the file pointed to by path is a GPIO chip character device.

Parameters

path	Path to check.

Returns

True if the file exists and is a GPIO chip character device or a symbolic link to it.

7.22 misc.hpp

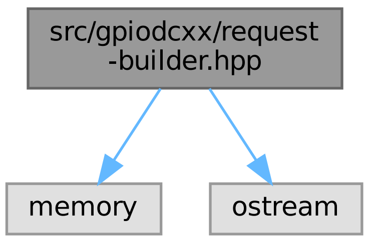
Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_MISC_HPP__
#define __LIBGPIOD_CXX_MISC_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <string>

namespace gpiod {

bool is_gpiochip_device(const ::std::filesystem::path& path);

const ::std::string& api_version();

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_MISC_HPP__ */
```

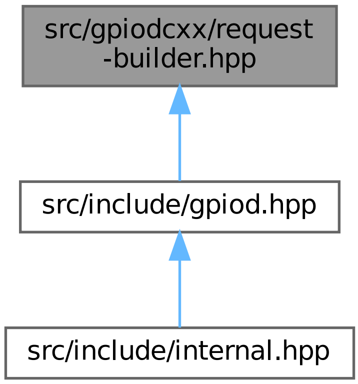
7.23 src/gpiodcxx/request-builder.hpp File Reference

```
#include <memory>
#include <ostream>
```

Include dependency graph for request-builder.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::request_builder

  Intermediate object storing the configuration for a line request.

Functions

- ::std::ostream & gpiod::operator<< (::std::ostream &out, const request_builder &builder)

  Stream insertion operator for GPIO request builder objects.

7.23.1 Function Documentation

7.23.1.1 operator<<()

```
GPIOD_CXX_API::std::ostream & gpiod::operator<< (
            ::std::ostream & out,
            const request_builder & builder )
```

Stream insertion operator for GPIO request builder objects.

Parameters

out	Output stream to write to.
builder	Request builder object to insert into the output stream.

Returns

Reference to out.

7.24 request-builder.hpp

Go to the documentation of this file.

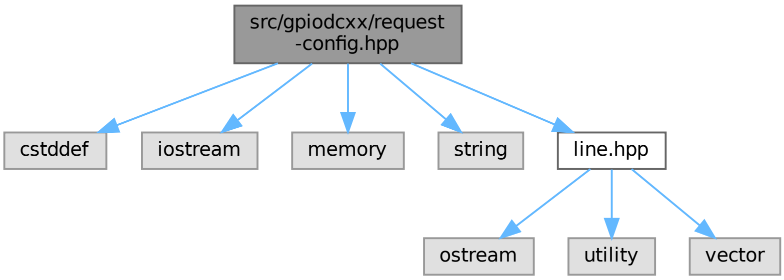
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_REQUEST_BUILDER_HPP__
#define __LIBGPIOD_CXX_REQUEST_BUILDER_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <memory>
#include <ostream>

namespace gpiod {

class chip;
class line_config;
class line_request;
class request_config;

class request_builder final
{
public:

    request_builder(const request_builder& other) = delete;

    request_builder(request_builder&& other) noexcept;

    ~request_builder();

    request_builder& operator=(const request_builder& other) = delete;

    request_builder& operator=(request_builder&& other) noexcept;

    request_builder& set_request_config(request_config& req_cfg);

    const request_config& get_request_config() const noexcept;

    request_builder& set_consumer(const ::std::string& consumer) noexcept;

    request_builder& set_event_buffer_size(::std::size_t event_buffer_size) noexcept;

    request_builder& set_line_config(line_config &line_cfg);

    const line_config& get_line_config() const noexcept;

    request_builder& add_line_settings(line::offset offset, const line_settings& settings);

    request_builder& add_line_settings(const line::offsets& offsets, const line_settings& settings);

    request_builder& set_output_values(const line::values& values);

    line_request do_request();

private:

    struct impl;

    request_builder(chip& chip);

    ::std::unique_ptr<impl> _m_priv;

    friend chip;
    friend ::std::ostream& operator«(::std::ostream& out, const request_builder& builder);
};

::std::ostream& operator«(::std::ostream& out, const request_builder& builder);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_REQUEST_BUILDER_HPP__ */
```

7.25 src/gpiodcxx/request-config.hpp File Reference

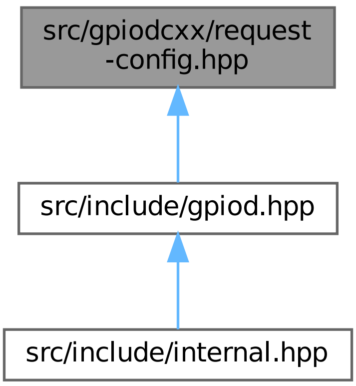
```
#include <cstddef>
#include <iostream>
```

```
#include <memory>
#include <string>
#include "line.hpp"
```

Include dependency graph for request-config.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::request_config

  Stores a set of options passed to the kernel when making a line request.

Functions

- ::std::ostream & gpiod::operator<< (::std::ostream &out, const request_config &config)

  Stream insertion operator for request_config objects.

7.25.1 Function Documentation

7.25.1.1 operator<<()

```
GPIOD_CXX_API::std::ostream & gpiod::operator<< (
            ::std::ostream & out,
            const request_config & config )
```

Stream insertion operator for request_config objects.

Parameters

out	Output stream to write to.
config	request_config to insert into the output stream.

Returns

Reference to out.

7.26 request-config.hpp

Go to the documentation of this file.

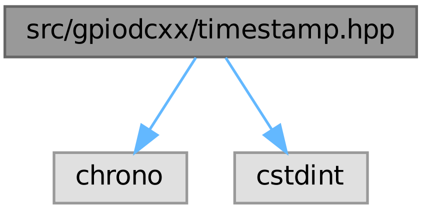
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_REQUEST_CONFIG_HPP__
#define __LIBGPIOD_CXX_REQUEST_CONFIG_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <cstddef>
#include <iostream>
#include <memory>
#include <string>

#include "line.hpp"

namespace gpiod {

class chip;

class request_config final
{
public:

    request_config();

    request_config(const request_config& other) = delete;

    request_config(request_config&& other) noexcept;

    ~request_config();

    request_config& operator=(request_config&& other) noexcept;

    request_config& set_consumer(const ::std::string& consumer) noexcept;

    ::std::string consumer() const noexcept;

    request_config& set_event_buffer_size(::std::size_t event_buffer_size) noexcept;

    ::std::size_t event_buffer_size() const noexcept;

private:

    struct impl;
```

```
    ::std::shared_ptr<impl> _m_priv;

    request_config& operator=(const request_config& other);

    friend request_builder;
};

::std::ostream& operator«(::std::ostream& out, const request_config& config);

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_REQUEST_CONFIG_HPP__ */
```

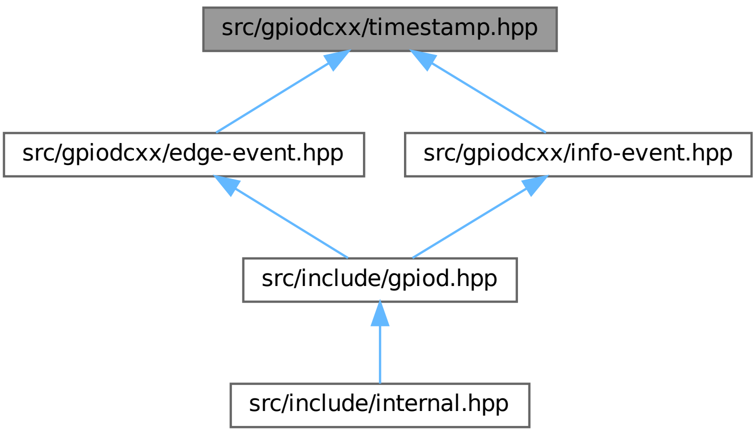
7.27 src/gpiodcxx/timestamp.hpp File Reference

```
#include <chrono>
#include <cstdint>
```

Include dependency graph for timestamp.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class gpiod::timestamp

  Stores the edge and info event timestamps as returned by the kernel and allows to convert them to std::chrono↩ ::time_point.

7.28 timestamp.hpp

Go to the documentation of this file.

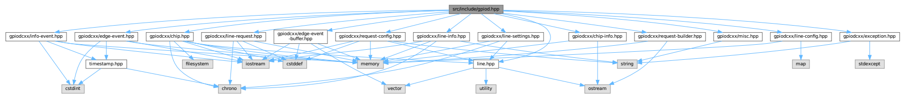
```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_TIMESTAMP_HPP__
#define __LIBGPIOD_CXX_TIMESTAMP_HPP__

#if !defined(__LIBGPIOD_GPIOD_CXX_INSIDE__)
#error "Only gpiod.hpp can be included directly."
#endif

#include <chrono>
#include <cstdint>

namespace gpiod {

class timestamp final
{
public:

    using time_point_monotonic = ::std::chrono::time_point<::std::chrono::steady_clock>;

    using time_point_realtime = ::std::chrono::time_point<::std::chrono::system_clock,
                                ::std::chrono::nanoseconds>;

    timestamp(::std::uint64_t ns) : _m_ns(ns) { }

    timestamp(const timestamp& other) noexcept = default;

    timestamp(timestamp&& other) noexcept = default;

    timestamp& operator=(const timestamp& other) noexcept = default;

    timestamp& operator=(timestamp&& other) noexcept = default;

    ~timestamp() = default;

    operator ::std::uint64_t() noexcept
    {
        return this->ns();
    }

    ::std::uint64_t ns() const noexcept
    {
        return this->_m_ns;
    }

    time_point_monotonic to_time_point_monotonic() const
    {
        return time_point_monotonic(::std::chrono::nanoseconds(this->ns()));
    }

    time_point_realtime to_time_point_realtime() const
    {
        return time_point_realtime(::std::chrono::nanoseconds(this->ns()));
    }

private:
    ::std::uint64_t _m_ns;
};

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_TIMESTAMP_HPP__ */
```

7.29 src/include/gpiod.hpp File Reference

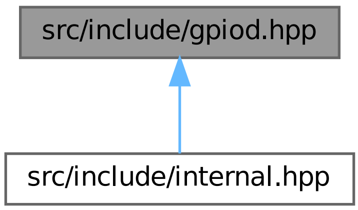
```
#include "gpiodcxx/chip.hpp"
#include "gpiodcxx/chip-info.hpp"
```

```
#include "gpiodcxx/edge-event.hpp"
#include "gpiodcxx/edge-event-buffer.hpp"
#include "gpiodcxx/exception.hpp"
#include "gpiodcxx/info-event.hpp"
#include "gpiodcxx/line.hpp"
#include "gpiodcxx/line-config.hpp"
#include "gpiodcxx/line-info.hpp"
#include "gpiodcxx/line-request.hpp"
#include "gpiodcxx/line-settings.hpp"
#include "gpiodcxx/misc.hpp"
#include "gpiodcxx/request-builder.hpp"
#include "gpiodcxx/request-config.hpp"
```

Include dependency graph for gpiod.hpp:



This graph shows which files directly or indirectly include this file:



7.30 gpiod.hpp

Go to the documentation of this file.

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_GPIOD_CXX_HPP__
#define __LIBGPIOD_GPIOD_CXX_HPP__

/*
 * We don't make this symbol private because it needs to be accessible by
 * the declarations in exception.hpp in order to expose the symbols of classes
 * inheriting from standard exceptions.
 */
#define GPIOD_CXX_API __attribute__((visibility("default")))

#define __LIBGPIOD_GPIOD_CXX_INSIDE__
#include "gpiodcxx/chip.hpp"
#include "gpiodcxx/chip-info.hpp"
#include "gpiodcxx/edge-event.hpp"
#include "gpiodcxx/edge-event-buffer.hpp"
#include "gpiodcxx/exception.hpp"
#include "gpiodcxx/info-event.hpp"
#include "gpiodcxx/line.hpp"
#include "gpiodcxx/line-config.hpp"
```

```
#include "gpiodcxx/line-info.hpp"
#include "gpiodcxx/line-request.hpp"
#include "gpiodcxx/line-settings.hpp"
#include "gpiodcxx/misc.hpp"
#include "gpiodcxx/request-builder.hpp"
#include "gpiodcxx/request-config.hpp"
#undef __LIBGPIOD_GPIOD_CXX_INSIDE__

#endif /* __LIBGPIOD_GPIOD_CXX_HPP__ */
```

7.31 internal.hpp

```
/* SPDX-License-Identifier: LGPL-2.1-or-later */
/* SPDX-FileCopyrightText: 2021-2022 Bartosz Golaszewski <brgl@bgdev.pl> */

#ifndef __LIBGPIOD_CXX_INTERNAL_HPP__
#define __LIBGPIOD_CXX_INTERNAL_HPP__

#include <gpiod.h>
#include <map>
#include <memory>
#include <string>
#include <utility>
#include <vector>

#include "gpiod.hpp"

namespace gpiod {

template<class cxx_enum_type, class c_enum_type>
cxx_enum_type get_mapped_value(c_enum_type value,
                   const ::std::map<c_enum_type, cxx_enum_type>& mapping)
{
    try {
        return mapping.at(value);
    } catch (const ::std::out_of_range& err) {
        /* FIXME Demangle the name. */
        throw bad_mapping(::std::string("invalid value for ") +
                  typeid(cxx_enum_type).name());
    }
}

void throw_from_errno(const ::std::string& what);
::gpiod_line_value map_output_value(line::value value);

template<class T, void F(T*)> struct deleter
{
    void operator()(T* ptr)
    {
        F(ptr);
    }
};

using chip_deleter = deleter<::gpiod_chip, ::gpiod_chip_close>;
using chip_info_deleter = deleter<::gpiod_chip_info, ::gpiod_chip_info_free>;
using line_info_deleter = deleter<::gpiod_line_info, ::gpiod_line_info_free>;
using info_event_deleter = deleter<::gpiod_info_event, ::gpiod_info_event_free>;
using line_settings_deleter = deleter<::gpiod_line_settings, ::gpiod_line_settings_free>;
using line_config_deleter = deleter<::gpiod_line_config, ::gpiod_line_config_free>;
using request_config_deleter = deleter<::gpiod_request_config, ::gpiod_request_config_free>;
using line_request_deleter = deleter<::gpiod_line_request, ::gpiod_line_request_release>;
using edge_event_deleter = deleter<::gpiod_edge_event, ::gpiod_edge_event_free>;
using edge_event_buffer_deleter = deleter<::gpiod_edge_event_buffer,
                      ::gpiod_edge_event_buffer_free>;

using chip_ptr = ::std::unique_ptr<::gpiod_chip, chip_deleter>;
using chip_info_ptr = ::std::unique_ptr<::gpiod_chip_info, chip_info_deleter>;
using line_info_ptr = ::std::unique_ptr<::gpiod_line_info, line_info_deleter>;
using info_event_ptr = ::std::unique_ptr<::gpiod_info_event, info_event_deleter>;
using line_settings_ptr = ::std::unique_ptr<::gpiod_line_settings, line_settings_deleter>;
using line_config_ptr = ::std::unique_ptr<::gpiod_line_config, line_config_deleter>;
using request_config_ptr = ::std::unique_ptr<::gpiod_request_config, request_config_deleter>;
using line_request_ptr = ::std::unique_ptr<::gpiod_line_request, line_request_deleter>;
using edge_event_ptr = ::std::unique_ptr<::gpiod_edge_event, edge_event_deleter>;
using edge_event_buffer_ptr = ::std::unique_ptr<::gpiod_edge_event_buffer,
                        edge_event_buffer_deleter>;

struct chip::impl
{
    impl(const ::std::filesystem::path& path);
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
```

```
    impl& operator=(impl&& other) = delete;

    void throw_if_closed() const;

    chip_ptr chip;
};

struct chip_info::impl
{
    impl() = default;
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    void set_info_ptr(chip_info_ptr& new_info);

    chip_info_ptr info;
};

struct line_info::impl
{
    impl() = default;
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    void set_info_ptr(line_info_ptr& new_info);

    line_info_ptr info;
};

struct info_event::impl
{
    impl() = default;
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    void set_info_event_ptr(info_event_ptr& new_event);

    info_event_ptr event;
    line_info info;
};

struct line_settings::impl
{
    impl();
    impl(const impl& other);
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    line_settings_ptr settings;
};

struct line_config::impl
{
    impl();
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    line_config_ptr config;
};

struct request_config::impl
{
    impl();
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    request_config_ptr config;
};

struct line_request::impl
{
    impl() = default;
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;
```

```

    void throw_if_released() const;
    void set_request_ptr(line_request_ptr& ptr);
    void fill_offset_buf(const line::offsets& offsets);

    line_request_ptr request;

    /*
     * Used when reading/setting the line values in order to avoid
     * allocating a new buffer on every call. We're not doing it for
     * offsets in the line & request config structures because they don't
     * require high performance unlike the set/get value calls.
     */
    ::std::vector<unsigned int> offset_buf;
};

struct edge_event::impl
{
    impl() = default;
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    virtual ~impl() = default;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    virtual ::gpiod_edge_event* get_event_ptr() const noexcept = 0;
    virtual ::std::shared_ptr<impl> copy(const ::std::shared_ptr<impl>& self) const = 0;
};

struct edge_event::impl_managed final : public edge_event::impl
{
    impl_managed() = default;
    impl_managed(const impl_managed& other) = delete;
    impl_managed(impl_managed&& other) = delete;
    ~impl_managed() = default;
    impl_managed& operator=(const impl_managed& other) = delete;
    impl_managed& operator=(impl_managed&& other) = delete;

    ::gpiod_edge_event* get_event_ptr() const noexcept override;
    ::std::shared_ptr<impl> copy(const ::std::shared_ptr<impl>& self) const override;

    edge_event_ptr event;
};

struct edge_event::impl_external final : public edge_event::impl
{
    impl_external();
    impl_external(const impl_external& other) = delete;
    impl_external(impl_external&& other) = delete;
    ~impl_external() = default;
    impl_external& operator=(const impl_external& other) = delete;
    impl_external& operator=(impl_external&& other) = delete;

    ::gpiod_edge_event* get_event_ptr() const noexcept override;
    ::std::shared_ptr<impl> copy(const ::std::shared_ptr<impl>& self) const override;

    ::gpiod_edge_event *event;
};

struct edge_event_buffer::impl
{
    impl(unsigned int capacity);
    impl(const impl& other) = delete;
    impl(impl&& other) = delete;
    impl& operator=(const impl& other) = delete;
    impl& operator=(impl&& other) = delete;

    int read_events(const line_request_ptr& request, unsigned int max_events);

    edge_event_buffer_ptr buffer;
    ::std::vector<edge_event> events;
};

} /* namespace gpiod */

#endif /* __LIBGPIOD_CXX_INTERNAL_HPP__ */
```

Index